# Innovative Evaluation Practices for BCSE308P -Computer Networks Lab

## Fall Semester 2026-27

# DESIGN AND IMPLEMENTATION OF A MULTI-THREADED HTTP/1.1 WEB SERVER FROM SCRATCH USING RAW TCP SOCKETS


**24BCE0702 YASH PRADHAN**

**24BCE0714 ANJINI PANDEY**


**B.Tech.**

*in*

**Computer Science and Engineering**

## School of Computer Science and Engineering




October 2026

# ABSTRACT

Concurrent web servers serve many TCP clients with a bounded thread pool and queue; when offered load exceeds capacity, queueing delay can dominate tail latency. This project implements a multithreaded HTTP/1.1 server in C17 using raw POSIX sockets and pthreads, with a bounded connection queue, four workers, a streaming parser for a documented subset of RFC 9112, routing, static-file delivery, metrics and a dashboard. It also implements Queue-and-Latency-Aware Admission Control (QLAAC), a three-state controller (NORMAL, CAUTION, OVERLOAD) that samples queue occupancy, worker utilisation, processing latency and queue-wait time each second. In CAUTION, keep-alive connections are limited to 16 requests; in OVERLOAD, new connections receive HTTP 503 with Retry-After. The baseline uses the same binary with the controller disabled. A paired benchmark used fresh server processes at 2, 8, 32 and 64 closed-loop clients, with five 10-second trials per configuration and standard clients (reconnect immediately after 503) and retry-aware clients (honour Retry-After). At 2 and 8 clients, QLAAC was comparable to baseline. At 32 clients, p99 latency fell 61.3% in both modes with no rejections, but throughput fell 13.7% and 13.4%, while p95 rose from about 1.3 ms to 73–75 ms. At 64 clients, standard-mode throughput fell 81.3%, 56.9% of attempts were rejected, and p99 improved only 9.4%. With retry-aware clients, p99 fell 60.6% but throughput fell 76.6% as clients backed off after Retry-After. These results show a latency–throughput trade-off rather than a universal improvement. The latest recorded suite reports 92 passing automated tests (44 C and 48 Python, including readiness-check regression tests), following the earlier 80-test run. Evaluation used one host, loopback traffic and a Python load generator; results should not be generalised to production.

# Certificate

This is to certify that the project report titled "Design and Implementation of a Multi-Threaded HTTP/1.1 Web Server from Scratch Using Raw TCP Sockets" is a bonafide record of the work done by **YASH PRADHAN (24BCE0702)** and **ANJINI PANDEY (24BCE0714)** in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P -Computer Networks Lab during the Fall Semester 2026-2027.

Name and Signature of the Students

# TABLE OF CONTENTS


| Sl.No | Contents | Page No. |
|---|---|---|
| | **Abstract** | 2 |
| **1.** | **INTRODUCTION** | 6 |
| | 1.1 Background and motivation | 6 |
| | 1.2 Problem statement | 6 |
| | 1.3 Objectives | 7 |
| | 1.4 Scope and limitations | 8 |
| | 1.5 Mapping to course modules and the lab experiment | 8 |
| **2.** | **Literature Survey** | 10 |
| | 2.1 Overview of the existing protocol or scheme | 10 |
| | 2.2 Review of related work | 10 |
| | 2.3 Research gap and justification of novelty | 11 |
| **3.** | **Proposed System** | 12 |
| | 3.1 System architecture or topology | 12 |
| | 3.2 Baseline mechanism description | 13 |
| | 3.3 Proposed mechanism: algorithm, pseudocode and flowchart | 13 |
| | 3.4 Novelty statement | 17 |
| | 3.5 Assumptions and design decisions | 17 |
| **4.** | **Implementation** | 18 |
| | 4.1 Tools, platform, versions and hardware | 18 |
| | 4.2 Simulation or testbed setup | 18 |
| | 4.3 Code structure and explanation of key modules | 20 |
| | 4.4 Challenges faced and how they were solved | 23 |
| **5.** | **Experimental Setup and Performance Evaluation** | 26 |
| | 5.1 Performance metrics and why they were chosen | 26 |
| | 5.2 Scenarios and parameter values | 27 |
| | 5.3 Results: tables and graphs, baseline vs proposed | 27 |
| | 5.4 Analysis and discussion | 32 |

| | | |
|---|---|---|
| | 5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails | 35 |
| **6.** | **Conclusion and Future Work** | 38 |
| | 6.1 Summary of work and key findings | 38 |
| | 6.2 Limitations | 39 |
| | 6.3 Future enhancements | 39 |
| **7.** | **References** | 40 |
| | References (IEEE style) | 40 |
| | Appendix A: Important source code or configuration listings | 41 |
| | Appendix B: Additional screenshots, packet captures, raw result tables | 44 |

# 1. INTRODUCTION

## 1.1 Background and motivation

The World Wide Web is built on a simple exchange: a client opens a TCP connection to a server, sends an HTTP request and receives a response. HTTP/1.1 is specified in RFC 9112 for message syntax and framing and in RFC 9110 for semantics [1], [2], while TCP provides the reliable, ordered byte stream on which the messages travel [3]. TCP delivers bytes, not messages: a single read on a socket can return part of a header, exactly one request, or several pipelined requests, so every HTTP server must buffer the stream and find message boundaries itself.

A server rarely serves one client at a time. Each accepted connection needs a thread of control for as long as the server is reading a request, producing a response or waiting on an idle persistent connection. Because threads, memory and file descriptors are finite, practical servers bound the resources they spend: a fixed-size pool of worker threads and a bounded queue of accepted-but-unserved connections. This bounded design protects the host, but it creates a queue. By Little's law, the mean number of requests in a stable system equals the arrival rate multiplied by the mean time in the system [11]. For a closed population of C clients, the mean response time therefore grows in proportion to C divided by the throughput the server can sustain, so once all workers are busy additional clients add waiting time rather than throughput. Long queues in front of a saturated resource are the cause of "bufferbloat" in network devices [9], and the resulting tail of slow requests matters disproportionately to users of large systems [10].

This project explores that relationship at the scale of a single machine. It builds a multi-threaded HTTP/1.1 server supporting a documented subset of the protocol directly on raw TCP sockets, with no third-party HTTP or event library, so that the implementation layers between the socket and the response are visible and measurable. It then adds an experimental overload controller, Queue-and-Latency-Aware Admission Control (QLAAC), and evaluates it against the same server running a fixed policy.

## 1.2 Problem statement

A thread-pool web server with a fixed number of workers W and a bounded connection queue of capacity Q has two failure modes under load. If clients hold persistent connections, each worker is occupied by one connection at a time, and connections beyond W wait in the queue.

Requests that arrive on a waiting connection see a delay equal to the service time of the connections ahead of them, so the tail of the latency distribution grows even though the median remains small. If the queue fills, new connections can only be refused. A fixed policy neither anticipates this nor distinguishes a harmless busy period from a developing overload.

The engineering problem addressed here is therefore twofold: (i) to implement a correct, bounded, concurrent HTTP/1.1 server on raw sockets; and (ii) to determine, with trustworthy measurements, whether a lightweight feedback controller that uses signals already available inside the server (worker utilisation, queue occupancy, processing latency and measured queue-wait time) can protect tail latency under overload, and at what cost in successful throughput, other latency percentiles and rejected requests.

### 1.3 Objectives

The following objectives were pursued and are implemented in the delivered code:

- Implement a TCP listener using the POSIX socket API (socket, bind, listen, accept, poll) with explicit handling of partial reads and writes.
- Implement a thread-safe bounded FIFO connection queue (mutex and condition variables) and a fixed pool of pthread workers that own each accepted connection.
- Implement a streaming parser for a documented subset of HTTP/1.1 (RFC 9112), with Content-Length framing, persistent connections, pipelined-byte preservation and explicit error statuses for unsupported features.
- Implement routing for a health endpoint, a JSON metrics endpoint, an echo endpoint and secure static-file delivery using sendfile(2) with path-traversal checks.
- Implement runtime telemetry (counters, rolling processing-latency and queue-wait reservoirs) and a browser dashboard that reads it.
- Design and implement QLAAC, a three-state queue-aware admission controller with hysteresis and keep-alive fair sharing, and compare it against a fixed-policy baseline (same binary, controller disabled).
- Build a benchmark harness whose accounting separates successful 2xx responses from other outcomes, offers a standard and a Retry-After-aware client mode, and evaluate both configurations with paired, interleaved trials on fresh server processes at several concurrency levels.
- Verify the implementation with an automated test suite and report limitations honestly.

### 1.4 Scope and limitations

Delivered: a Linux/POSIX server in C17 (3,569 lines in src/ and 1,144 lines in include/ in the source snapshot listed in Table 4.3), GET, HEAD and POST handling within the limits described in Chapter 4, Content-Length-framed request bodies up to 10 MiB, persistent connections with a configurable request limit and idle timeout, a bounded queue and four-worker pool, QLAAC with queue-wait telemetry, metrics, a dashboard, and a current recorded suite of 92 automated tests (44 C unit tests and 48 Python tests, including the 12 readiness-check tests in Appendix B.8).

**Outside the scope or not implemented:** chunked request decoding (requests with Transfer-Encoding: chunked receive 501 Not Implemented), Expect: 100-continue (417 Expectation Failed), TLS/HTTPS, HTTP/2 and HTTP/3, compression, caching validators and range requests, virtual hosts, authentication, and event-driven I/O (epoll). RFC 9112 §6.1 requires HTTP/1.1 recipients to be able to parse chunked transfer coding (defined in §7.1), so returning 501 for chunked requests is an explicit implementation limitation and deviation from full HTTP/1.1 conformance. Full RFC 9112 compliance, production-grade security, and scalability beyond the tested range are not claimed; the evidence supports only the behaviours named here. The performance evaluation is limited to one workload (GET /health), one host per experiment, and four concurrency levels (Section 5.2).

### 1.5 Mapping to course modules and the lab experiment

Because an official BCSE308P syllabus or lab-experiment specification document was not archived within the project repository, specific curriculum module numbers and official laboratory experiment titles could not be independently verified from repository files and are not fabricated here. Instead, Table 1.1 formally maps each implemented system component to foundational Computer Networks concepts (including transport-layer socket abstractions, application-layer HTTP framing, thread-pool concurrency models, queueing dynamics, and empirical performance evaluation) that the implementation concretely exercises.

**Table 1.1:** Mapping of project components to Computer Networks concepts

| Computer Networks concept | Where it appears in the project |
|---|---|
| Transport layer: TCP connection lifecycle and byte-stream semantics [3] | src/tcp_socket.c: listen backlog, accept, partial recv/send loops with poll deadlines, shutdown(SHUT_WR) and lingering close |
| Application layer: HTTP/1.1 message syntax, framing and persistent connections [1], [2] | src/http_parser.c, src/http_response.c: request line and header parsing, Content-Length framing, keep-alive, 400/405/417/501/505-class errors |
| Client–server model and concurrency | src/thread_pool.c, src/connection_queue.c: acceptor thread, bounded FIFO, fixed worker pool |
| Queueing delay, buffering and overload (cf. [9], [11]) | Bounded queue (capacity 64), enqueue-to-dequeue queue-wait telemetry, QLAAC thresholds, 503 with Retry-After [2] |
| Network performance measurement | benchmark/run_experiment.py, run_reproducible_evaluation.py: throughput, latency percentiles, status-class accounting, paired trials |
| Network application security | src/static_files.c: traversal rejection and canonical-root check; parser size limits; tests/test_security.py |

# 2. LITERATURE SURVEY

## 2.1 Overview of the existing protocol or scheme

**HTTP/1.1.** RFC 9112 defines the HTTP/1.1 message format: a request line, header fields, an optional body delimited by Content-Length or chunked transfer coding, and rules for persistent connections, in which several request/response exchanges reuse one TCP connection unless either side sends "Connection: close" [1]. RFC 9110 defines method and status semantics, including the 503 (Service Unavailable) status, which signals temporary overload or maintenance, and the Retry-After header field that a server may send with it [2]. Neither document prescribes how a server should decide when to refuse work; that policy is left to implementations.

**TCP.** RFC 9293 consolidates the TCP specification: a connection-oriented, reliable, ordered byte stream with flow control [3]. A server calls listen with a backlog, and the kernel completes the three-way handshake for connections that sit in the accept queue until the application calls accept. This is why, in the experiments, a client can observe a connection as established while the server application has not yet picked it up.

**Server architectures and overload handling.** Thread-per-connection and thread-pool servers are simple but block a thread per active connection. Event-driven designs multiplex many connections on few threads, and hybrids such as staged event-driven architecture (SEDA) split request processing into stages joined by bounded queues, with per-stage controllers that adapt resource use and perform admission control [4], [5]. Comparative studies show that careful tuning matters as much as architecture [7]. Under overload, scheduling approaches such as shortest-remaining-processing-time ordering of connections can improve response times without refusing work [6], whereas admission-control approaches deliberately refuse some work in order to protect the rest [5]. In networking, active queue management such as CoDel uses queueing delay rather than queue length as the control signal, because long standing queues, not bursts, are harmful [8], [9].

## 2.2 Review of related work

Table 2.1 summarises the standards and prior work most relevant to this project. The "Gaps" column states what each item does not provide with respect to the present problem, not a criticism of its own goals.

**Table 2.1:** Literature survey

| S.No | Authors, year | Technique | Metrics and tool | Merits | Gaps |
|---|---|---|---|---|---|
| 1 | Fielding, Nottingham, Reschke (Eds.), 2022 [1], [2] | HTTP/1.1 message syntax and framing; HTTP semantics, 503 and Retry-After | Normative specification (no experimental metrics) | Authoritative definition of framing, persistent connections and overload status codes | Does not prescribe an overload or admission policy for servers |
| 2 | Welsh, Culler and Brewer, 2001 [4] | SEDA: stages connected by bounded queues with dynamic resource controllers | Throughput and response time of a SEDA-based web server compared with conventional servers | Explicit queues make load visible; per-stage control | Requires a staged event architecture; far more complex than a flat thread pool |
| 3 | Welsh and Culler, 2003 [5] | Adaptive overload control: admission control driven by measured response time of each stage | Response-time percentile against a target; SEDA-based services | Feedback on measured latency; request-level load shedding | Depends on the SEDA framework and stage instrumentation |
| 4 | Schroeder and Harchol-Balter, 2006 [6] | Size-based (SRPT-style) connection scheduling for static content under overload | Mean response time under overload; web-server testbed | Improves response time without refusing requests | Needs known response sizes; does not bound queue or protect against saturation |
| 5 | Pariag et al., 2007 [7] | Comparison of thread-per-connection, event-driven and hybrid server architectures | Throughput and response time under load; tuned server implementations | Shows tuning, not architecture alone, determines performance | Does not study admission control or overload shedding |
| 6 | Nichols and Jacobson, 2012 [8]; Gettys and Nichols, 2011 [9] | Delay-based active queue management (CoDel); analysis of excessive buffering | Queueing delay (sojourn time) in packet buffers; simulation and real traffic | Control on delay rather than length; motivates measuring delay | Targets packet queues in routers, not application connection queues |

## 2.3 Research gap and justification of novelty

**Baseline limitation.** A thread-pool server with a fixed bounded queue and a fixed keep-alive policy treats every load level identically. In the paired measurements of this project, the baseline sustained its successful throughput (about 6.8–6.9 thousand 2xx responses per second) as the number of clients grew from 8 to 64, but the p99 latency of successful responses grew from 28.5 ms to 434.8 ms in standard mode (28.4 ms to 444.2 ms in retry-aware mode), because connections waited behind pinned workers (Section 5.4). The queue never overflowed in the baseline trials (no 5xx response in any of the 40 baseline trials; the deepest queue seen in the diagnostic timelines was 60 of 64), so the only protection it offers (a 503 when the queue is full) never activated. Established admission-control frameworks [4], [5] address such behaviour but are built around staged architectures and are heavy for a teaching-scale server.

**What this project adds.** The contribution is a measured C implementation combining established ideas: a three-state controller using worker, queue, latency and queue-wait signals; keep-alive fair sharing (at most 16 requests in CAUTION); accept-time 503/Retry-After rejection in OVERLOAD; recovery hysteresis; and paired benchmarking that separately counts 2xx responses, 503s, timeouts, transport errors and client back-off.

# 3. PROPOSED SYSTEM

## 3.1 System architecture or topology

Figure 3.1 shows the request path. A single listener thread polls the listening socket (100 ms poll timeout so it can notice shutdown), accepts connections and consults the QLAAC state. In OVERLOAD it writes a fixed 503 response with Retry-After: 5 and closes the connection itself. Otherwise it pushes the connection onto a bounded FIFO queue without blocking, time-stamping it on entry; if the queue is full it writes a 503 with Retry-After: 5. Four pre-spawned worker threads pop connections, and each worker keeps its connection until it ends. When a worker dequeues a connection it records the enqueue-to-dequeue wait (the queue wait) in the metrics collector. Within that connection the worker repeatedly parses a request with the streaming parser, dispatches it through the router (health, metrics, echo or static file) and sends the response, then records latency and status. A separate sampler thread wakes every 1000 ms, reads queue occupancy, worker utilisation, the rolling p95 processing latency and the rolling p95 queue wait, and updates the QLAAC state.

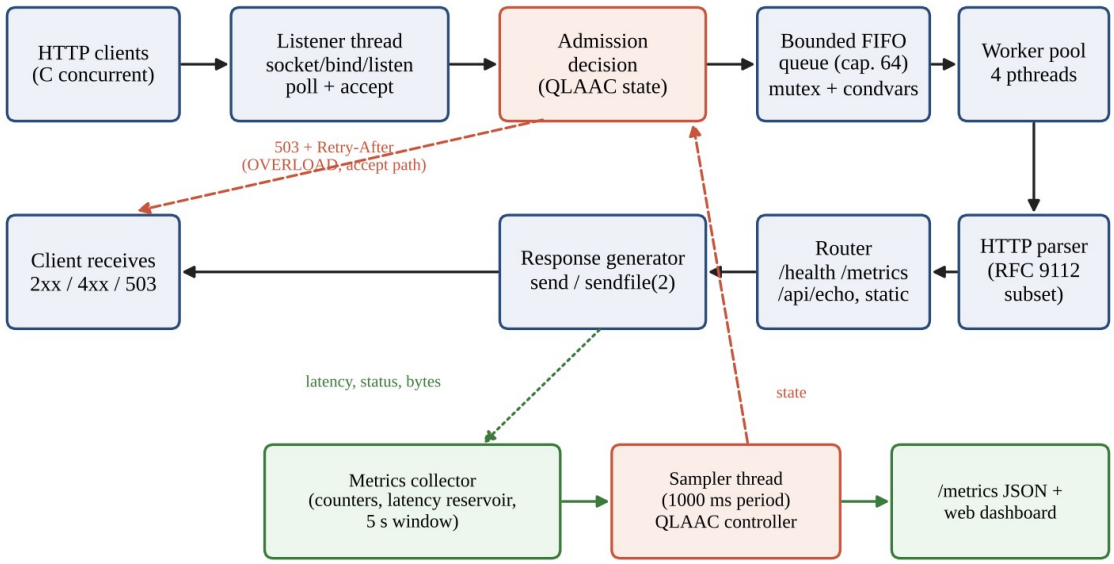



**Figure 3.1:** Request-handling architecture: client, listener, admission decision, bounded queue, workers, parser, router, response generator, and the metrics/controller feedback path (drawn from src/server.c, thread_pool.c and the architecture description). Queue-wait samples are recorded by the workers into the same metrics collector (not drawn separately).

### 3.2 Baseline mechanism description

The baseline is **the same executable with adaptive_control_enabled = false** (the default). The controller update is never called, adaptive_controller_evaluate always returns ADMIT, and no keep-alive throttling occurs. The baseline therefore uses a fixed policy:

- 4 worker threads (server_config_init_defaults); connection queue capacity 64; listen backlog 128.
- Persistent connections allowed for up to 100 requests per connection and 5 s idle timeout.
- If the bounded queue is full, the listener replies 503 (Retry-After: 5) and closes. This path is part of the baseline; it never triggered in the baseline trials (5xx count 0 in all 40 baseline trials of the paired evaluation).
- No latency feedback, no early shedding, no keep-alive adaptation.

The baseline is **not** an unbounded queue. An earlier summary in the project package described it that way, but src/connection_queue.c enforces the capacity limit and the harness never exceeded it: with at most 64 clients, at most 60 connections can wait while four are in service.

### 3.3 Proposed mechanism: algorithm, pseudocode and flowchart

QLAAC runs in the sampler thread once per sampling window (1000 ms). It receives the queue occupancy ratio q (current depth divided by capacity 64), the worker utilisation u (active workers divided by 4), the p95 processing latency and the p95 queue wait. Both p95 values are computed from 1,024-slot reservoirs restricted to the last 5 seconds. The processing latency of a request is measured in the worker from the moment request bytes are available to the completion of the send. The queue wait of a connection is measured from the moment it was enqueued after accept until a worker dequeues it, and is recorded at dequeue time. The controller derives the flags below from the thresholds in Table 3.1 and then transitions between the states shown in Figure 3.2.

**Table 3.1:** QLAAC signals and thresholds (defaults in server_config_init_defaults and config/server.conf.example)

| Signal / parameter | Threshold | Meaning |
|---|---|---|
| Worker utilisation u | caution ≥ 0.85; saturated ≥ 1.00 | Active workers divided by total workers; counts only together with queue or latency evidence |
| Queue occupancy q | growing ≥ 0.70; critical ≥ 0.90 | Queue depth divided by capacity (64) |
| Processing latency p95 | elevated ≥ 50 ms; critical ≥ 150 ms | p95 of request latency (request bytes available to send complete) over the last 5 s |
| Queue-wait p95 | elevated ≥ 25 ms; critical ≥ 100 ms | p95 of enqueue-to-dequeue wait over the last 5 s; counted only while q ≥ 0.20 |
| Sustained-pressure windows | 3 | Consecutive windows of compound pressure before OVERLOAD |
| Recovery windows | 5 | Consecutive healthy windows to step down one state (server.c passes recovery_sample_count = 5; the controller's own fallback of 3 applies only if that field is 0) |
| Sampling period | 1000 ms | Controller evaluation interval |

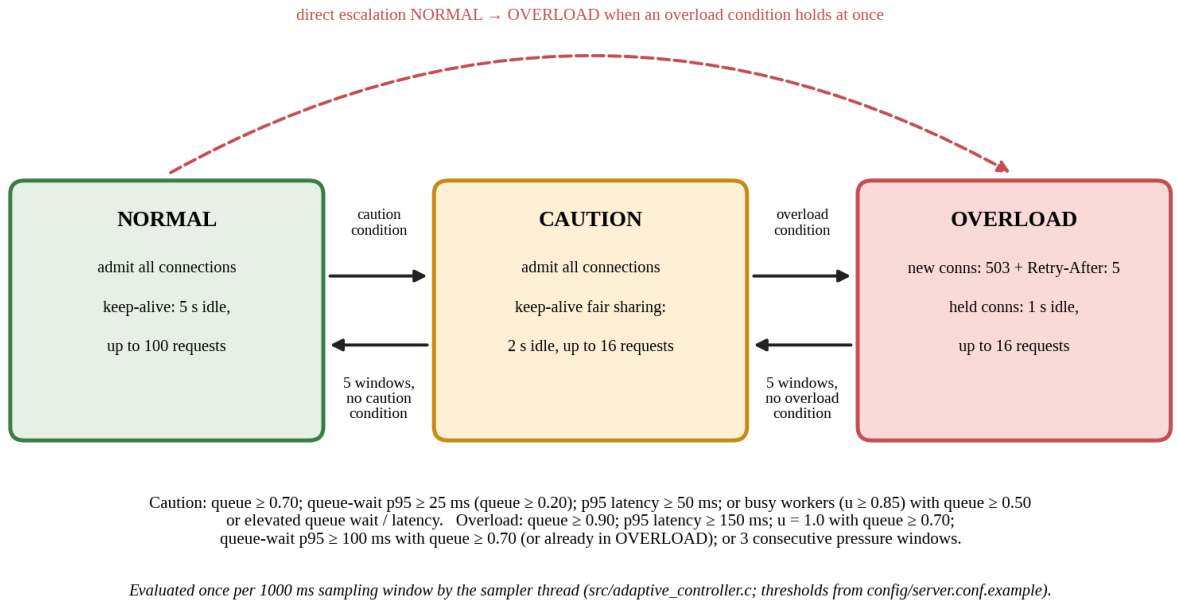



**Figure 3.2:** QLAAC state machine (from src/adaptive_controller.c and src/thread_pool.c). CAUTION limits keep-alive connections to 16 requests; OVERLOAD rejects new connections at accept time and applies the same limit to held connections.

**Decision rules** (adaptive_controller_update_queue_aware). Define worker_saturated = (u ≥ 1.0), worker_caution = (u ≥ 0.85), queue_growing = (q ≥ 0.70), queue_critical = (q ≥ 0.90), latency_elevated / latency_critical = (p95 processing latency ≥ 50 / 150 ms) and, only while q ≥ 0.20, queue_wait_elevated / queue_wait_critical = (p95 queue wait ≥ 25 / 100 ms). A window counts toward sustained pressure when (saturated and q ≥ 0.50) or (queue_growing and (queue_wait_elevated or latency_elevated)). The overload condition holds if queue_critical, or latency_critical, or (queue_wait_critical and (queue_growing or the state is already OVERLOAD)), or (saturated and queue_growing), or the sustained-pressure counter has

reached 3. The caution condition holds if the overload condition holds or any of queue_growing, queue_wait_elevated, queue_wait_critical, latency_elevated holds, or workers are at least at the caution level ($u \geq 0.85$) while $q \geq 0.50$ or queue wait or latency is elevated.

**Moderate queue pressure versus overload.** Worker saturation alone, with a short queue and low latency, leaves the controller in NORMAL: it is the normal operating point of a busy but healthy pool. A long queue wait with a moderate queue is treated as head-of-line delay behind persistent connections rather than overload: queue_wait_critical triggers OVERLOAD only when the queue is also growing ($q \geq 0.70$) or the controller is already in OVERLOAD (the latter prevents premature recovery); otherwise it raises CAUTION. For example, with 32 clients and four workers about 28 connections wait ($q = 0.44$) while processing latency is sub-millisecond, and the controller enters CAUTION, not OVERLOAD. With 64 clients about 60 connections wait ($q = 0.94 \geq 0.90$), which is critical queue occupancy and leads to OVERLOAD.

**Actuation.** In CAUTION each worker, before reading the next request on a connection, sets its keep-alive idle timeout to 2 s and its request limit to min(configured limit, 16); the connection is closed after the 16th response (a Connection: close response), the client reconnects, and the new connection joins the back of the queue. This is keep-alive fair sharing: persistence is shortened, not disabled. In OVERLOAD the listener answers every newly accepted connection with "503 Service Unavailable", "Retry-After: 5" and "Connection: close" without queuing it, and workers apply the same 16-request limit with a 1 s idle timeout to connections they already hold; held connections are not interrupted mid-stream. Worker threads obtain the state through adaptive_controller_get_state(), a side-effect-free query; only the listener's call to adaptive_controller_evaluate() counts shed decisions.

**Recovery.** From OVERLOAD, five consecutive windows in which the overload condition does not hold are required to step down to CAUTION; from CAUTION, five consecutive windows with no flag raised are required to return to NORMAL. Escalation is immediate (a single window), de-escalation is slow, which prevents oscillation but also prolongs rejection (Section 5.4).

**Pseudocode** (one sampling window):

```
function QLAAC_UPDATE(q, u, lat95, qw95):            # once per 1000 ms window
    sat   <- (u >= 1.00);   wcaut <- (u >= 0.85)
    qgrow <- (q >= 0.70);   qcrit <- (q >= 0.90)
    lelev <- (lat95 >= 50); lcrit <- (lat95 >= 150)  # ignored if lat95 = 0
    qwel  <- (qw95 >= 25) and (q >= 0.20)            # queue-wait signals count only
    qwcr  <- (qw95 >= 100) and (q >= 0.20)           # while the queue is non-trivial
    cand  <- (sat and q >= 0.50) or (qgrow and (qwel or lelev))
    pressure <- cand ? pressure + 1 : 0
    over  <- qcrit or lcrit or (qwcr and (qgrow or state == OVERLOAD))
             or (sat and qgrow) or pressure >= 3
    caut  <- over or qgrow or qwel or qwcr or lelev
             or ((wcaut or sat) and (q >= 0.50 or qwel or lelev))
    if over:
        state <- OVERLOAD ; healthy <- 0
    elif caut:
        if state == OVERLOAD: healthy++ ; if healthy >= 5: state <- CAUTION; healthy <- 0
        elif state == NORMAL: state <- CAUTION ; healthy <- 0
        else: healthy <- 0
    else:
        if state == OVERLOAD: healthy++ ; if healthy >= 5: state <- CAUTION; healthy <- 0
        elif state == CAUTION: healthy++ ; if healthy >= 5: state <- NORMAL; healthy <- 0
        else: state <- NORMAL
```

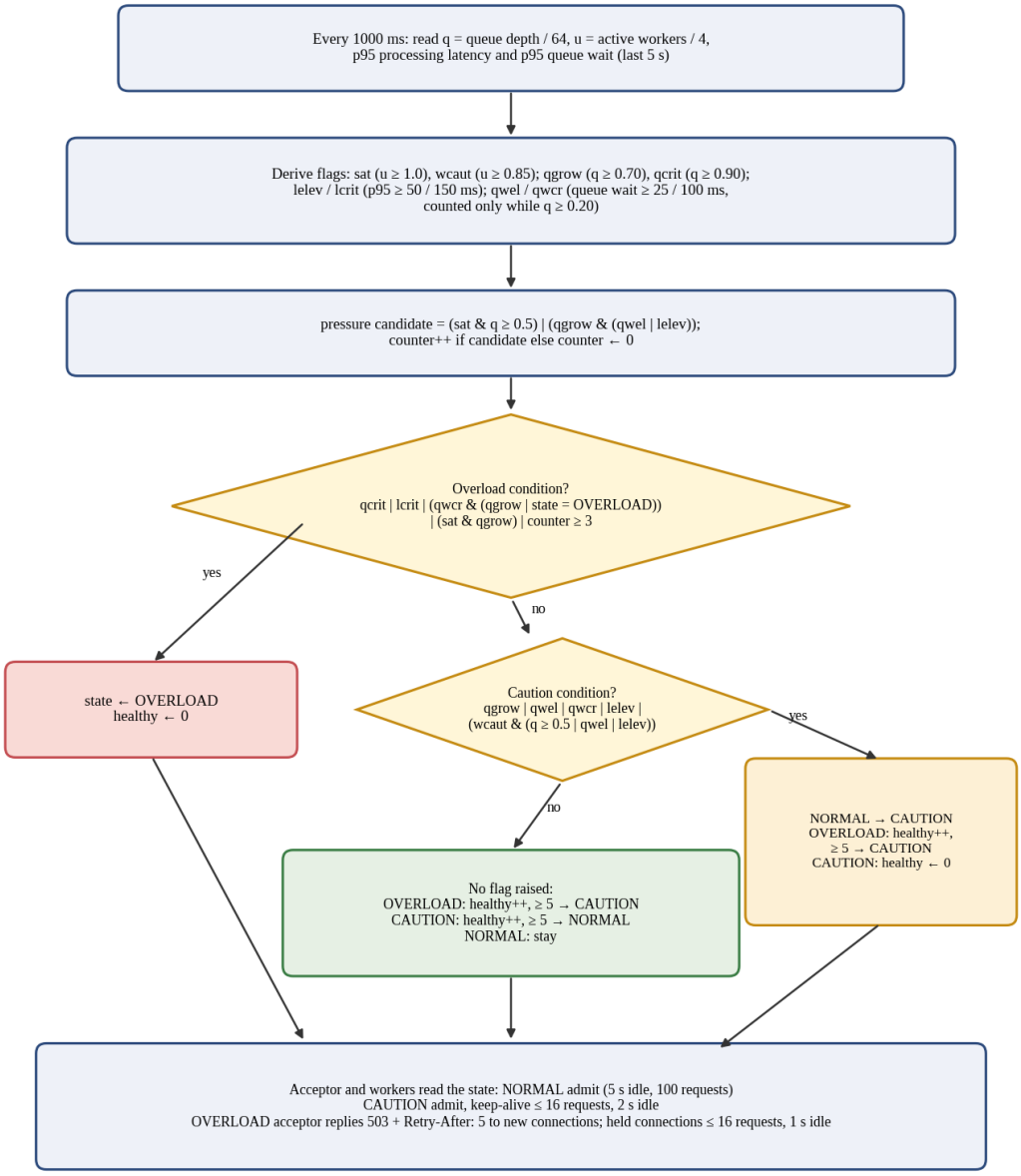



**Figure 3.3:** Flowchart of one QLAAC evaluation window (condensed from adaptive_controller_update_queue_aware).

### 3.4 Novelty statement

QLAAC differs from the baseline in exactly four implemented ways: (1) a feedback loop (sampler thread plus controller, fed by processing-latency and queue-wait telemetry) that the baseline does not run; (2) keep-alive fair sharing for connections in service, whereas the baseline always allows up to 100 requests and 5 s idle per connection (CAUTION: at most 16 requests and 2 s idle; OVERLOAD: at most 16 requests and 1 s idle); (3) accept-time 503 rejection with Retry-After: 5 in OVERLOAD, before the connection occupies a queue slot, whereas the baseline rejects only when the queue is completely full; and (4) hysteresis on recovery. Everything else (worker count, queue capacity, parser, router, response generation) is identical because it is the same binary. The controller is a composition of established ideas (Section 2.3) and is evaluated as an engineering mechanism, not presented as a new class of algorithm.

### 3.5 Assumptions and design decisions

**Table 3.2:** Design decisions and assumptions (values verified in the source and configuration files)

| Decision | Value | Rationale / assumption |
|---|---|---|
| Worker threads | 4 | Default in server.c; fixed at start. Assumed small to make saturation reachable on a laptop-class host; not tuned (sizing study not performed). |
| Queue capacity | 64 connections | Bounds memory; equals the highest client count tested, so the baseline queue cannot overflow in closed-loop tests with $C \leq 64$ (at most 60 connections wait). |
| Listen backlog | 128 | Kernel accept queue sized above the application queue. |
| Keep-alive (NORMAL) | 100 requests, 5 s idle | Defaults; allows connection reuse. The harness itself closes a connection after 50 requests. |
| Keep-alive (CAUTION) | ≤ 16 requests, 2 s idle | Rotates connections so that waiting connections get a turn without disabling persistence. The value 16 was chosen during development (an earlier revision used 32); no sensitivity study was done. |
| Keep-alive (OVERLOAD, held connections) | ≤ 16 requests, 1 s idle | Same request cap with a shorter idle timeout. |
| Sampling window | 1000 ms | Coarse relative to request times (about 0.1 ms); a window can contain thousands of requests. |
| Latency / queue-wait window and reservoir | 5 s / 1,024 samples each | metrics.h constants; the p95 values reflect only recent samples. |
| Overload response | 503 + Retry-After: 5 (accept gate and queue-full path) | Semantics of RFC 9110 [2]. |
| Measurement points | Processing latency: request bytes available to send complete. Queue wait: enqueue to dequeue. | Queue wait is recorded only when a worker dequeues the connection, so connections still waiting contribute no sample yet (Section 5.5). |

**Configuration note.** The v4 benchmark explicitly loaded config/server.conf.example, so its 8,192-byte header and I/O-buffer limits applied instead of the compiled 16,384-byte defaults.

# 4. IMPLEMENTATION

## 4.1 Tools, platform, versions and hardware

**Table 4.1:** Implementation environment

| Item | Value | Source / status |
|---|---|---|
| Language and standard | C17 (-std=c17), POSIX.1-2008 (-D_POSIX_C_SOURCE=200809L), pthreads | Makefile |
| Compiler warnings | -Wall -Wextra -Wpedantic -Wformat=2 -Wshadow and others | Makefile |
| OS-specific features | Linux sendfile(2) for static files | src/tcp_socket.c |
| Libraries | libc and POSIX only; no third-party HTTP or event library | Source inspection |
| Benchmark harness | Python 3 standard library only (socket, threading); closed-loop clients; paired runner starts a fresh server process per trial | benchmark/run_experiment.py, benchmark/run_reproducible_evaluation.py |
| Source revision | Branch fix/qlaac-queue-aware-control; latest application commit d3ae7d92a33c0ea3b314fca08be73ffc6cbba2ac | Stated by the authors; the supplied ZIP has no .git directory, so the hash was not independently verified |
| Benchmark platform (Chapter 5, v4 runs) | Linux under WSL2 on a Windows 11 host, loopback 127.0.0.1 | docs/qlaac-controller-revision.md. CPU model, kernel and compiler of the v4 runs are not recorded in the CSV files |
| Earlier workstation audit (previous report revision) | Intel Core Ultra 7 155H (22 vCPUs), 7.5 GiB RAM, Linux 6.18.33.2 WSL2, GCC 15.2.0, Python 3.14.4 | Reported by the authors in the previous revision; not re-verified here and not linked to the v4 CSV files |
| Report-revision verification sandbox (October 2026) | Ubuntu 24.04, Linux 6.18.44, 1 logical CPU, GCC 13.3.0, Python 3.12.3 | Used only to run the tests (Appendix B.2) and recompute the statistics; not used for any benchmark |

The v4 benchmark results in Chapter 5 were produced on the original development platform (a Windows 11 host running Linux under WSL2 with loopback networking). The CPU model, kernel and compiler of those runs were not written into the CSV headers, and no CPU temperature or clock-frequency data were collected. The sandbox in the last row of Table 4.1 was used only to execute the automated tests and to recompute the benchmark statistics from the raw CSV files; it is not comparable with, and was not used for, the benchmark measurements.

## 4.2 Simulation or testbed setup

The testbed is a single host: the server listens on 127.0.0.1:8080 and the Python load generator runs on the same machine, so no physical network or wire latency is involved. No network emulation was used. Every trial runs against a freshly started server process (baseline: bin/server; QLAAC: bin/server -a), so controller state, hysteresis counters and latency reservoirs cannot carry over between trials. After the server accepts connections, the runner

performs a 1 s, 2-client warm-up, pauses 0.2 s and then starts the timed 10 s trial. The harness opens C threads; each thread repeatedly opens a TCP connection, sends up to 50 sequential GET requests with "Connection: keep-alive", records the client-observed latency of each response (from just before sendall to the last body byte; TCP connect time and any back-off wait are not included), closes the connection and opens a new one until the trial duration elapses. If a response carries "Connection: close" the thread reconnects. The workload is closed-loop: a client issues its next request only after the previous response. The two client modes differ only after a 5xx response. Standard mode reconnects immediately. Retry-aware mode waits for the number of seconds in the Retry-After header (1 s if absent or invalid) before reconnecting, and records the time spent waiting as total_backoff_sec.

**Table 4.2:** Testbed and experiment parameters (from benchmark/run_experiment.py, benchmark/run_reproducible_evaluation.py and the raw CSV files)

| Parameter | Value |
|---|---|
| Target endpoint | GET /health (small JSON body of 89 bytes) |
| Concurrency levels (clients C) | 2, 8, 32, 64 |
| Trials | 5 matched pairs (trial index t = 1–5) per concurrency level and client mode: one baseline and one QLAAC trial of 10 s each, 40 trials per client mode, 80 in total |
| Execution order | Within each pair the order alternates: odd trials run the baseline first, even trials run QLAAC first; every trial uses a fresh server process |
| Configurations | Baseline: QLAAC disabled; Proposed: QLAAC enabled (bin/server -a) |
| Server settings | bin/server -c <abs path>/config/server.conf.example -p 8080 [-a]: 4 workers, queue 64, backlog 128, keep-alive 100 requests / 5 s (NORMAL); QLAAC thresholds as in Table 3.1 |
| Client modes | Standard: reconnect immediately after any response, including 503. Retry-aware: after a 5xx, wait Retry-After seconds (1 s if missing or invalid) before reconnecting |
| Client behaviour | Up to 50 requests per connection; 2 s connect/read timeout; 0.5 s pause after each trial |
| Data files | results/raw/experiment_{baseline,qlaac}_{standard,retry-aware} _v4_paired_20261019.csv (20 rows each); diagnostic_timeline_*_v4_paired_20261019.csv (controller state, queue depth and queue-wait p95, polled about every 0.25 s) |
| Benchmark execution command | python3 benchmark/run_reproducible_evaluation.py --client-mode {standard\| retry-aware} (paired mode is the default; 10 s, 5 trials, C = 2, 8, 32, 64) |

The earlier sequential harness ran all baseline trials and then all QLAAC trials on one long-lived server process. Because execution order and elapsed time were confounded with the policy, and controller state could carry over between trials, the final evaluation uses the paired runner described above. Earlier datasets (sequential v2/v3 runs and the previous "tuned" runs) were produced with earlier controller versions or with the sequential harness; they are not used for the conclusions of this report and are listed with the reasons in Appendix B.5 and B.6.

## 4.3 Code structure and explanation of key modules

Table 4.3: Source modules (line counts from the archived source snapshot; readiness-check additions are described in Appendix B.8)

| File | Lines | Responsibility |
|---|---|---|
| src/main.c | 140 | Command-line flags (-c, -p, -t, -d, -a), configuration loading, signal handling (SIGINT/SIGTERM), startup banner |
| src/server.c | 420 | Defaults and config-file parser; poll accept loop; signal-safe shutdown; accept-gate QLAAC admission and 503 shedding (Retry-After: 5); queue push and queue-full 503; sampler thread forwarding p95 processing latency and p95 queue wait to the controller |
| src/tcp_socket.c | 486 | Listener creation (getaddrinfo, SO_REUSEADDR), non-blocking accept, poll-based recv/send with deadlines, zero-copy sendfile with offset-preserving fallback, lingering close |
| src/connection_queue.c | 194 | Thread-safe bounded circular FIFO; non-blocking push that time-stamps the connection (enqueue); timed pop; shutdown broadcast |
| src/thread_pool.c | 285 | Fixed pthread pool; per-connection keep-alive loop; request timing; queue-wait recording at dequeue; side-effect-free state query and keep-alive fair sharing (16 requests) in CAUTION and OVERLOAD; worker activity count; stop flag check |
| src/http_parser.c | 479 | Streaming request parser: request line and headers (max 64); URI ≤ 2,047 B; decimal Content-Length; Host and duplicate-Host validation; rejects TE+CL and repeated Content-Length; bare-CR rejection plus deliberate bare-LF rejection; local header limits; unsupported chunked/Expect handling |
| src/http_response.c | 332 | Status phrases, standard headers (Date, Server, Connection), error pages, body/file response sending, resource leak cleanup |
| src/router.c | 205 | Dispatch: /health, /metrics (JSON), /api/echo (POST), static files; query-string path separation |
| src/static_files.c | 287 | MIME types, segment-aware traversal checks, URL percent-decoding (NUL/backslash rejection), realpath canonical-root validation with symlink O_NOFOLLOW protections |

| File | Lines | Responsibility |
|---|---|---|
| src/adaptive_controller.c | 299 | QLAAC state machine with queue-aware transitions, hysteresis, admission decision, shed counter, side-effect-free state query, thread-safe snapshotting |
| src/metrics.c | 318 | Atomic-style counters under lock, 1,024-sample processing-latency and queue-wait reservoirs over a 5 s window, p50/p95/p99 snapshots, JSON export |
| src/util.c | 124 | Monotonic time, RFC 1123 date formatting, logging |
| public/index.html | 1,470 | Single-page dashboard (HTML/CSS/JavaScript) polling /metrics |
| tests/ (C: 5 files, Python: 5 files; current suite) | 1,478 C + 889 Python | The current recorded suite contains 44 C unit tests and 48 Python unit/integration tests (92 total). Appendix B.8 records the complete passing run and readiness-check regression coverage. |
| benchmark/*.py (6 scripts) | 1,808 | run_experiment.py (489, load generator, standard and retry-aware modes), run_reproducible_evaluation.py (561, paired runner), verify_benchmark_statistics.py (187), plotting and diagnostic helpers |

Listener and TCP handling. The listener uses a non-blocking socket and poll with an async-signal-safe shutdown flag. tcp_recv_some and tcp_send_all loop until the requested number of bytes has moved or a monotonic deadline passes, handling short reads and writes explicitly. Static files are sent with a zero-copy Linux sendfile loop with dynamic lseek(SEEK_CUR) offset preservation in the user-space fallback path. After a response is complete the worker calls shutdown(SHUT_WR) and waits briefly for the peer before closing (tcp_lingering_close), preventing unread client data from generating connection resets.

**Queue and thread pool.** The queue is a circular buffer guarded by one mutex and two condition variables. The listener pushes with a zero timeout, so a full queue is detected immediately and the connection is rejected with 503 instead of blocking the accept loop; each queued connection carries an enqueue timestamp. Workers pop with a configurable timeout (worker_idle_timeout_sec, defaulting to 2 s), compute the enqueue-to-dequeue wait, record it in the metrics collector, and check an atomic stop flag so they terminate promptly upon shutdown even with persistent sockets open. A worker owns its socket exclusively for the lifetime of the connection and tracks active worker utilisation.

Parser, routing and static files. The parser accumulates bytes in a per-connection buffer, returns “need more data” until headers and any Content-Length body are complete, and preserves extra bytes for the next pipelined request. It requires a Host field and rejects duplicate Host fields (RFC 9112 §3.2). It rejects requests containing both Transfer-Encoding and Content-Length with 400 Bad Request and closes the connection (RFC 9112 §6.3, item 3). It rejects every repeated Content-Length field, including identical values, as a conservative policy permitted by RFC 9110 §8.6; invalid or conflicting message framing is rejected with 400 as applicable under RFC 9112 §6.3, item 5. Content-Length values use decimal digits only (RFC 9110 §8.6 and RFC 9112 §6.2); signs and overflow are rejected, and bodies above the configured 10 MiB maximum are rejected. Chunked requests return 501 and Expect: 100-continue returns 417; the absence of chunked request decoding is an explicit limitation relative to RFC 9112 §6.1 and §7.1. The parser rejects bare CR outside content; it also deliberately rejects bare LF as a defensive choice, although RFC 9112 §2.2 permits recipients to recognize a single LF as a line terminator. Whitespace before a field colon is rejected under RFC 9112 §5.1. The 128-byte header-name limit, 2,048-byte header-value limit, and 8,192-byte configured / 16,384-byte compiled-default header-block limits are local operational bounds, not numeric limits prescribed by the RFCs; oversized header fields/blocks receive 431. The router separates query strings from request targets before dispatching /health, /metrics, /api/echo, and static files. Static files use segment-aware traversal checks, reject malformed percent escapes, NUL bytes (%00) and backslash paths, and validate the canonical root with realpath() and O_NOFOLLOW / fstat device/inode protections.

**Metrics, controller queries and dashboard.** Each request records status class, processing latency, bytes received and bytes sent; each dequeued connection records its queue wait. 503 responses written by the listener increment separate overload counters and are deliberately not counted as completed requests. Worker threads query QLAAC state via adaptive_controller_get_state(), a pure side-effect-free query that avoids extraneous shed counting. The /metrics route (excluded from latency statistics) returns counters, queue depth, active workers, rolling throughput, processing-latency and queue-wait percentiles and a thread-safe snapshot of QLAAC state as JSON. Figure 4.1 shows the dashboard reading these values; it was captured before the queue-wait display was added to public/index.html (Appendix B.1).

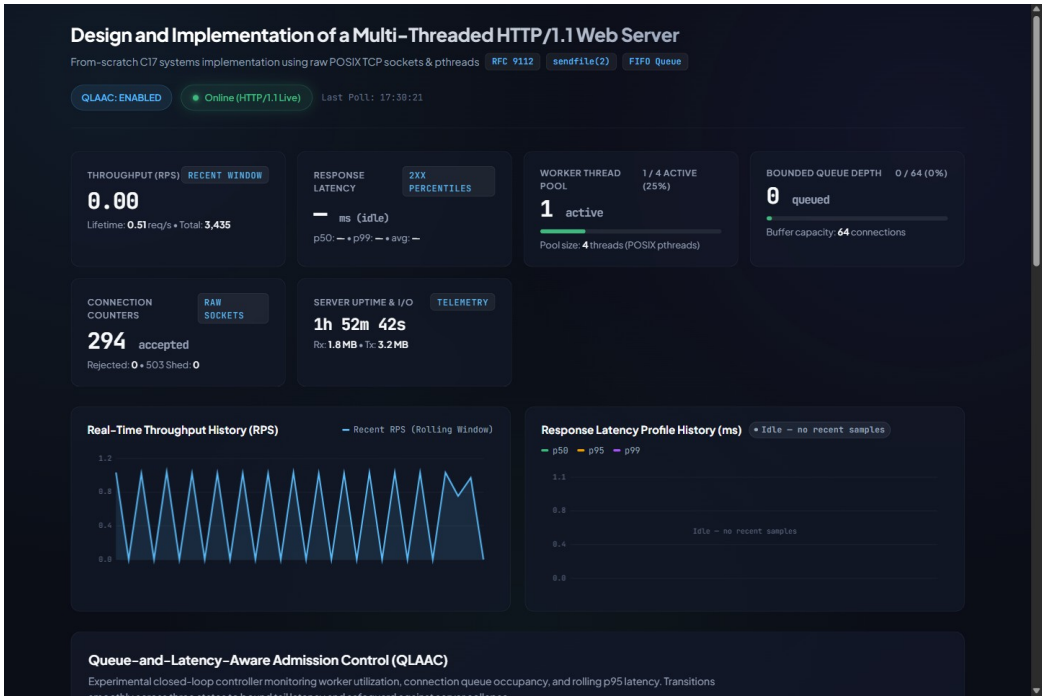



**Figure 4.1:** Live dashboard at idle (QLAAC enabled, 4 workers, queue 0/64, no recent latency samples), captured from the running server. Further captures are in Appendix B.1.

## 4.4 Challenges faced and how they were solved

**Table 4.4:** Challenges, solutions and evidence

| Challenge | Solution | Evidence |
|---|---|---|
| TCP gives a byte stream; one read may contain a partial or several requests | Per-connection buffer, streaming parser with a "need more data" state and preservation of pipelined leftover bytes | test_parser.c: test_pipelined_leftover_preservation; benchmark harness uses the same leftover-buffer rule |
| Early benchmark scripts counted any response beginning "HTTP/1.1" as success, so 503 rejections inflated throughput | Harness rewritten to parse the status line, enforce Content-Length framing and count 2xx, 3xx, 4xx, 5xx, timeouts, transport errors and invalid responses separately; throughput uses only 2xx | docs/experiment-methodology.md section 7; legacy CSVs kept but excluded (Appendix B.5); test_benchmark_accounting.py |
| Percentiles are undefined when a trial has no 2xx response | Reported as N/A instead of 0.00 ms | test_zero_success_reports_na_latency; N/A handling in run_experiment.py |
| A 503 written by the listener is not a processed request | Separate overload counters; "completed" is not incremented; a failed 503 write is not counted as sent | test_server_metrics.c: overload accounting and failed-write tests |
| QLAAC first collapsed throughput or shed heavily at moderate load (C = 8 and 32) | Controller revised in steps. (1) CAUTION no longer reduces keep-alive to 1 request but caps it at 16, and worker saturation alone no longer raises CAUTION. (2) Queue wait is measured from enqueue to dequeue and used as a signal. (3) Critical queue wait with a moderate queue ($q < 0.70$) now raises CAUTION, not OVERLOAD. Verified effect at C = 32: 0 rejections in all 10 v4 trials. | docs/qlaac-controller-revision.md; docs/qlaac-final-diagnostic.md; tests/test_controller.c (includes the moderate-queue case); Appendix B.4 and B.6 |

| Challenge | Solution | Evidence |
|---|---|---|
| Server-dependent tests skip silently when no HTTP listener is active | The earlier test harness recorded an 80-test run (44 C, 36 Python) in Appendix B.2. The later readiness-check regression tests are described in Appendix B.8, which records the current 92-test suite and its passing output. | Historical run: Appendix B.2. Current recorded suite: Appendix B.8; tests/test_readiness_check.py. |
| QLAAC shed counter overcount due to worker evaluation calls | Resolved in current source: Worker loops now query controller state via adaptive_controller_get_state() (pure, side-effect-free query without state modification or counter increments). The server telemetry counters have distinct physical semantics: (1) qlaac_shed_requests counts QLAAC admission-rejection decisions made at the accept gate; (2) total_connections_rejected counts both QLAAC shed decisions and bounded-queue-full rejections; and (3) total_overload_503_sent counts successful server-side writes of HTTP 503 responses across all rejection paths. Because queue-full rejections add successful 503 writes without incrementing qlaac_shed_requests, and client connections that close or reset before or during the server's HTTP 503 response write prevent total_overload_503_sent from incrementing, there is no guaranteed ordering between the QLAAC shed counter and the successful-503-write counter. | src/thread_pool.c, src/server.c, src/adaptive_controller.c; test_qlaac_get_state_no_side_effects; live reconciliation in test_reconciliation_overload_and_client_5xx_timeouts |
| Directory traversal attacks, encoding bypasses, and symlink escapes | Segment-aware path traversal parser; URL percent-decoding with strict rejection of malformed escapes, NUL bytes (%00), and backslash paths; canonical root sandboxing with realpath() and symlink O_NOFOLLOW / fstat dev/inode protections. | src/static_files.c; integration tests in tests/test_security.py |
| Malformed framing, signed Content-Length, header smuggling, and descriptor leaks | Uses decimal-only Content-Length parsing (rejecting signs and overflow); rejects TE+CL and every repeated Content-Length field (including identical values) as a conservative policy allowed by RFC 9110 §8.6; rejects invalid/conflicting framing under RFC 9112 §6.3. Rejects bare CR and deliberately rejects bare LF (RFC 9112 §2.2 permits a recipient to recognize a single LF). Rejects whitespace before a field colon (RFC 9112 §5.1). The 128-byte name, 2,048-byte value, and 8,192/16,384-byte total-header limits are local choices, not numeric RFC limits; | src/http_parser.c, src/http_response.c, src/tcp_socket.c; tests/test_parser.c, tests/test_responses.c; RFC 9112 §§2.2, 3.2, 5.1, 6.1–6.3, 7.1; RFC 9110 §§5.4, 8.6 |

| Challenge | Solution | Evidence |
|---|---|---|
| | oversized headers receive 431. Chunked requests return 501, an explicit limitation against RFC 9112 §6.1/§7.1. Error paths clean up buffers and descriptors; sendfile fallback preserves its offset. | |
| Sequential benchmark runs gave inconsistent baselines (C = 2: 21,653 vs 13,149 req/s for identical code) | Paired runner: baseline and QLAAC alternate order within each matched pair, each trial on a fresh server process. Cause of the earlier drift is attributed to execution order or environment; no temperature or frequency data exist to identify it further. | benchmark/ run_reproducible_evaluation.py; v3 vs v4 CSV files; Appendix B.6 |
| Queue wait before a worker picks up a connection was invisible to the controller | Enqueue and dequeue timestamps; queue-wait reservoir and p95 passed to the controller | src/connection_queue.c, src/thread_pool.c, src/metrics.c; test_server_metrics.c |

**Testing.** The current recorded suite contains 44 C unit tests and 48 Python unit/integration tests, for 92 total. The recorded run of ./scripts/run_all_tests.sh in Appendix B.8 reports all 92 tests passing. The earlier 80-test run (44 C and 36 Python) remains documented as a historical execution in Appendix B.2; the readiness-check regression tests were added afterward. The tests cover controller transitions and hysteresis, the moderate-queue case that must not trigger OVERLOAD, side-effect-free state queries and thread-safe snapshots, framing validation, signed Content-Length, bare CR and deliberate bare-LF rejection, sendfile offset fallback integrity, queue capacity, metrics accounting, query-string routing, graceful worker shutdown, segment-aware path-traversal prevention, URL percent-decoding and symlink sandboxing. The C test suite was also run under sanitizers as recorded in Appendix B.2; that sanitizer evidence covers the C unit tests, not every runtime path or the Python suite. These are targeted functional/security regression tests, not performance measurements, and they do not establish full RFC compliance or production security.

# 5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

## 5.1 Performance metrics and why they were chosen

**Table 5.1:** Metrics and their definitions

| Metric | Definition (from the harness) | Why chosen |
|---|---|---|
| Successful response throughput (2xx responses per second) | success_2xx divided by the 10 s trial duration; 503s, other errors and timeouts excluded | Measures useful work; prevents rejected requests from inflating throughput |
| Latency p50, p95, p99 (ms) | Per-trial percentile of client-observed latency of 2xx responses only, index int(p·(n−1)) of the sorted samples; from just before sending the request to the last body byte. Excludes TCP connect time and client back-off. Reported as the mean of the five per-trial percentiles | Median shows typical cost; p95/p99 show the tail that overload control targets |
| Attempted requests per trial | Requests sent by all client threads (including requests answered 503) | Offered-load indicator. Not fixed: in a closed loop it depends on server speed and, in retry-aware mode, on client back-off |
| HTTP 503 rejections (request shedding) | http_5xx per trial (all 5xx were 503). Rejection rate = mean http_5xx per trial ÷ mean attempted requests per trial | Cost of protection. Attempted equals completed responses in all 80 trials (no timeouts, transport errors or invalid responses) |
| Client back-off time (s) | total_backoff_sec: time slept after 5xx responses, summed over all client threads within one trial (client-thread-seconds; at most C × 10 s). Retry-aware mode only | Separates waiting time from response latency; indicates reduced offered load |
| Timeouts, transport errors, invalid responses | Counts per trial; zero in all 80 trials | Detect failures other than deliberate 503s |
| Controller state, queue depth, queue-wait p95 | Polled from /metrics about every 0.25 s into the diagnostic timeline CSV files | Explains mechanisms; qualitative evidence only, not part of the statistics |

**Which requests enter each statistic.** Latency percentiles describe only requests that returned 2xx. When QLAAC rejects traffic (C = 64), the percentiles describe the admitted requests only (a form of survivorship) and must be read together with the rejection rate and the throughput. Attempted requests are not an independent offered-load measure: in standard mode clients reconnect immediately after a rejection, so rejections inflate attempts; in retry-aware mode clients that honour Retry-After sleep and therefore offer less load. Means and standard deviations are computed across the five trial-level values of each metric (sample SD, n − 1); percentiles are computed per trial and then averaged, not pooled.

## 5.2 Scenarios and parameter values

**Table 5.2:** Scenarios used in this report (v4 paired CSV files; 5 trials × 10 s per configuration and client mode)

| Scenario | Clients C | Relation to the 4 workers | Observed QLAAC state (diagnostic timelines, both client modes) |
|---|---|---|---|
| S1 (low load) | 2 | C ≤ workers; no waiting | NORMAL in every polled sample; no 503 |
| S2a (medium load) | 8 | C = 2 × workers; 4 connections wait | NORMAL in every polled sample; no 503 |
| S2b (medium-high load) | 32 | C = 8 × workers; 28 connections wait (queue 44% full) | CAUTION from about 0.7 s to the end of the trial; no 503 |
| S3 (high load) | 64 | C = 16 × workers; 60 connections wait (queue 94% full) | OVERLOAD from about 1 s; recovery to CAUTION after about 5 s, then OVERLOAD again in most trials; 503s |

The faculty template provides one row for medium load; two medium points (C = 8 and C = 32) are reported because they behave differently under the baseline (p99 of 28.5 ms against 201.9 ms in standard mode) and under QLAAC (NORMAL against CAUTION). Datasets that were not used in Section 5.3, with the reasons, are listed in Appendix B.5. All values below were recomputed independently from the four raw v4 CSV files and agree with benchmark/verify_benchmark_statistics.py (--v4-standard and --v4-retryaware).

## 5.3 Results: tables and graphs, baseline vs proposed

Table 5.3 follows the official VIT results-table structure, comparing the baseline and proposed QLAAC mechanism for each client mode, concurrency scenario and metric. It reports the mean ± sample SD across five trial-level observations and a direction-normalized improvement percentage. Table 5.4 retains the separate signed-change and outcome accounting for attempts, HTTP 503 responses, rejection rate and client back-off.

**Table 5.3:** Baseline versus proposed results in the VIT template format (mean ± sample SD across five paired 10-second trials)

| Scenario | Metric | Baseline (mean ± SD) | Proposed (mean ± SD) | Improvement (%) |
|---|---|---|---|---|
| S1 Low (C=2) — Standard | Successful 2xx throughput (req/s) | 13,806.78 ± 752.14 | 13,230.22 ± 566.31 | −4.2% |
| S1 Low (C=2) — Standard | p50 latency (ms) | 0.11 ± 0.01 | 0.12 ± 0.01 | −5.5% |
| S1 Low (C=2) — Standard | p95 latency (ms) | 0.26 ± 0.02 | 0.28 ± 0.03 | −6.9% |
| S1 Low (C=2) — Standard | p99 latency (ms) | 0.43 ± 0.03 | 0.46 ± 0.06 | −7.4% |
| S2 Medium (C=8) — Standard | Successful 2xx throughput (req/s) | 6,869.56 ± 196.87 | 6,989.02 ± 270.05 | +1.7% |
| S2 Medium (C=8) — Standard | p50 latency (ms) | 0.53 ± 0.02 | 0.52 ± 0.02 | +2.6% |
| S2 Medium (C=8) — Standard | p95 latency (ms) | 1.30 ± 0.05 | 1.29 ± 0.06 | +0.9% |

| Scenario | Metric | Baseline (mean ± SD) | Proposed (mean ± SD) | Improvement (%) |
|---|---|---|---|---|
| S2 Medium (C=8) — Standard | p99 latency (ms) | 28.54 ± 0.72 | 28.02 ± 1.01 | +1.8% |
| S2b Med-high (C=32) — Standard | Successful 2xx throughput (req/s) | 6,938.70 ± 249.64 | 5,988.46 ± 101.62 | −13.7% |
| S2b Med-high (C=32) — Standard | p50 latency (ms) | 0.52 ± 0.02 | 0.64 ± 0.01 | −21.8% |
| S2b Med-high (C=32) — Standard | p95 latency (ms) | 1.31 ± 0.06 | 72.99 ± 1.25 | −5471.8% |
| S2b Med-high (C=32) — Standard | p99 latency (ms) | 201.93 ± 6.43 | 78.17 ± 1.19 | +61.3% |
| S3 High (C=64) — Standard | Successful 2xx throughput (req/s) | 6,922.30 ± 150.62 | 1,295.26 ± 44.72 | −81.3% |
| S3 High (C=64) — Standard | p50 latency (ms) | 0.53 ± 0.01 | 0.65 ± 0.03 | −23.5% |
| S3 High (C=64) — Standard | p95 latency (ms) | 1.33 ± 0.03 | 13.74 ± 0.52 | −931.8% |
| S3 High (C=64) — Standard | p99 latency (ms) | 434.75 ± 9.41 | 393.73 ± 21.75 | +9.4% |
| S1 Low (C=2) — Retry-aware | Successful 2xx throughput (req/s) | 14,136.06 ± 1,867.40 | 14,115.42 ± 856.21 | −0.1% |
| S1 Low (C=2) — Retry-aware | p50 latency (ms) | 0.11 ± 0.02 | 0.11 ± 0.01 | +5.3% |
| S1 Low (C=2) — Retry-aware | p95 latency (ms) | 0.25 ± 0.03 | 0.25 ± 0.02 | 0.0% |
| S1 Low (C=2) — Retry-aware | p99 latency (ms) | 0.41 ± 0.04 | 0.42 ± 0.04 | −3.9% |
| S2 Medium (C=8) — Retry-aware | Successful 2xx throughput (req/s) | 6,884.42 ± 78.47 | 6,587.70 ± 219.70 | −4.3% |
| S2 Medium (C=8) — Retry-aware | p50 latency (ms) | 0.53 ± 0.00 | 0.55 ± 0.01 | −3.4% |
| S2 Medium (C=8) — Retry-aware | p95 latency (ms) | 1.29 ± 0.03 | 1.39 ± 0.11 | −7.1% |
| S2 Medium (C=8) — Retry-aware | p99 latency (ms) | 28.40 ± 0.22 | 29.49 ± 0.83 | −3.8% |
| S2b Med-high (C=32) — Retry-aware | Successful 2xx throughput (req/s) | 6,767.70 ± 270.04 | 5,859.92 ± 91.19 | −13.4% |
| S2b Med-high (C=32) — Retry-aware | p50 latency (ms) | 0.54 ± 0.02 | 0.65 ± 0.01 | −20.8% |
| S2b Med-high (C=32) — Retry-aware | p95 latency (ms) | 1.34 ± 0.07 | 74.65 ± 1.24 | −5454.6% |
| S2b Med-high (C=32) — Retry-aware | p99 latency (ms) | 206.97 ± 7.90 | 80.15 ± 1.84 | +61.3% |
| S3 High (C=64) — Retry-aware | Successful 2xx throughput (req/s) | 6,805.68 ± 116.05 | 1,591.48 ± 528.46 | −76.6% |
| S3 High (C=64) — Retry-aware | p50 latency (ms) | 0.54 ± 0.01 | 0.59 ± 0.02 | −10.4% |
| S3 High (C=64) — Retry-aware | p95 latency (ms) | 1.33 ± 0.05 | 29.11 ± 45.62 | −2095.6% |
| S3 High (C=64) — Retry-aware | p99 latency (ms) | 444.17 ± 7.13 | 175.15 ± 47.31 | +60.6% |

Note: “Improvement (%)” is direction-normalized against the baseline. For successful throughput, it is (QLAAC − baseline) / baseline × 100; for latency percentiles, it is (baseline − QLAAC) / baseline × 100. Positive values indicate improvement and negative values indicate degradation for the metric being reported. Table 5.4 separately reports signed change from baseline. Interpret all metrics jointly because a lower latency percentile does not by itself imply better overall performance. Percentages are calculated from unrounded trial means; displayed means and standard deviations are rounded for readability.

**Table 5.4:** Change from baseline to QLAAC and outcome accounting (means over 5 trials; rejection rate = mean 503s ÷ mean attempts)

| Client mode | C | Δ 2xx req/s | Δ p50 | Δ p95 | Δ p99 | Attempts / trial (baseline → QLAAC) | 503 / trial | Rej. rate (%) | Backoff (s) |
|---|---|---|---|---|---|---|---|---|---|
| Standard | 2 | −4.2% | +5.5% | +6.9% | +7.4% | 138,068 → 132,302 | 0.0 | 0.00 | n/a |
| | 8 | +1.7% | −2.6% | −0.9% | −1.8% | 68,696 → 69,890 | 0.0 | 0.00 | n/a |
| | 32 | −13.7% | +21.8% | +5472% | −61.3% | 69,387 → 59,885 | 0.0 | 0.00 | n/a |
| | 64 | −81.3% | +23.5% | +932% | −9.4% | 69,223 → 30,079 | 17,126.8 | 56.94 | n/a |
| Retry-aware | 2 | −0.1% | −5.3% | +0.0% | +3.9% | 141,361 → 141,154 | 0.0 | 0.00 | 0.00 |
| | 8 | −4.3% | +3.4% | +7.1% | +3.8% | 68,844 → 65,877 | 0.0 | 0.00 | 0.00 |
| | 32 | −13.4% | +20.8% | +5455% | −61.3% | 67,677 → 58,599 | 0.0 | 0.00 | 0.00 |
| | 64 | −76.6% | +10.4% | +2096% | −60.6% | 68,057 → 16,043 | 128.0 | 0.80 | 511.91 |

**Rejection rate and back-off time.** The rejection rate is the mean number of 503 responses per trial divided by the mean number of attempted requests per trial. In the retry-aware C = 64 case every trial recorded exactly 128 rejections and the mean attempt count was 16,042.8, giving 128 / 16,042.8 = 0.80% (equivalently 640 / 80,214 over all five trials). Averaging the five per-trial ratios gives 0.86% because the trials with fewer attempts (minimum 12,099) weigh more; the 0.86% figure in the repository diagnostic uses that convention. This report uses 0.80% throughout. The back-off of 511.91 s is the mean over the five trials of total_backoff_sec, which sums the time slept by all 64 client threads within one 10 s trial (per-trial values 541.82, 539.38, 492.30, 445.90 and 540.14 s). It is therefore measured in client-thread-seconds, not wall-clock seconds, not summed across trials and not per client; the maximum possible is 64 × 10 = 640 client-thread-seconds, so clients spent about 80% of their available time waiting. The number 128 equals 2 × 64, which is consistent with each thread being rejected twice, but per-thread logs were not recorded to confirm this.

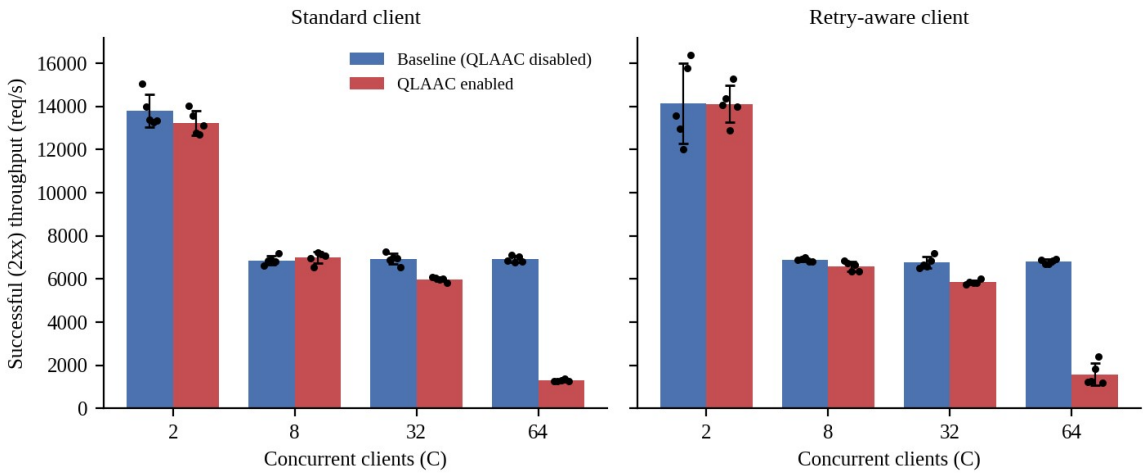



**Figure 5.1:** Successful (2xx) throughput, mean ± SD over 5 paired trials; dots are individual trials. Left: standard client; right: retry-aware client.

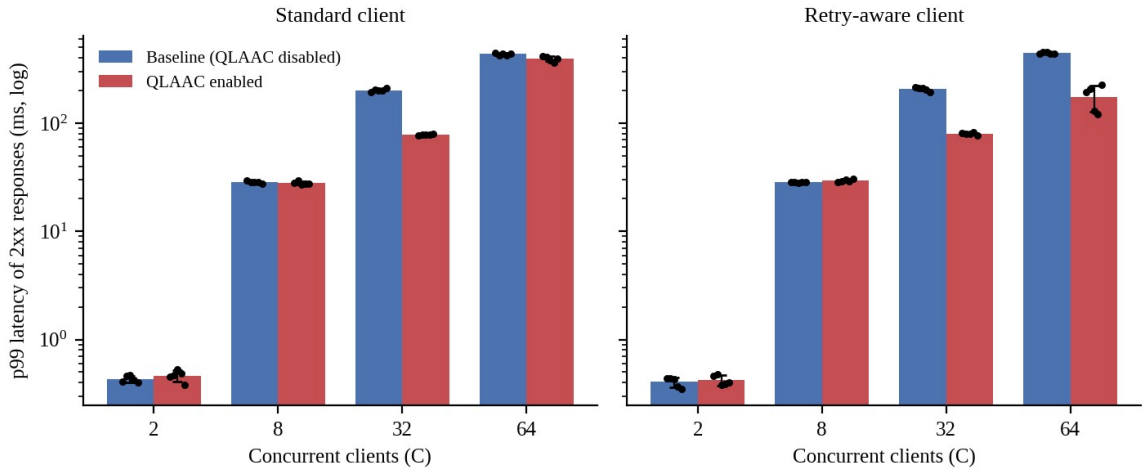



**Figure 5.2:** p99 latency of 2xx responses (log scale), mean ± SD; dots are individual trials. At C = 64 in retry-aware mode QLAAC p99 ranges from 121 to 225 ms across trials.

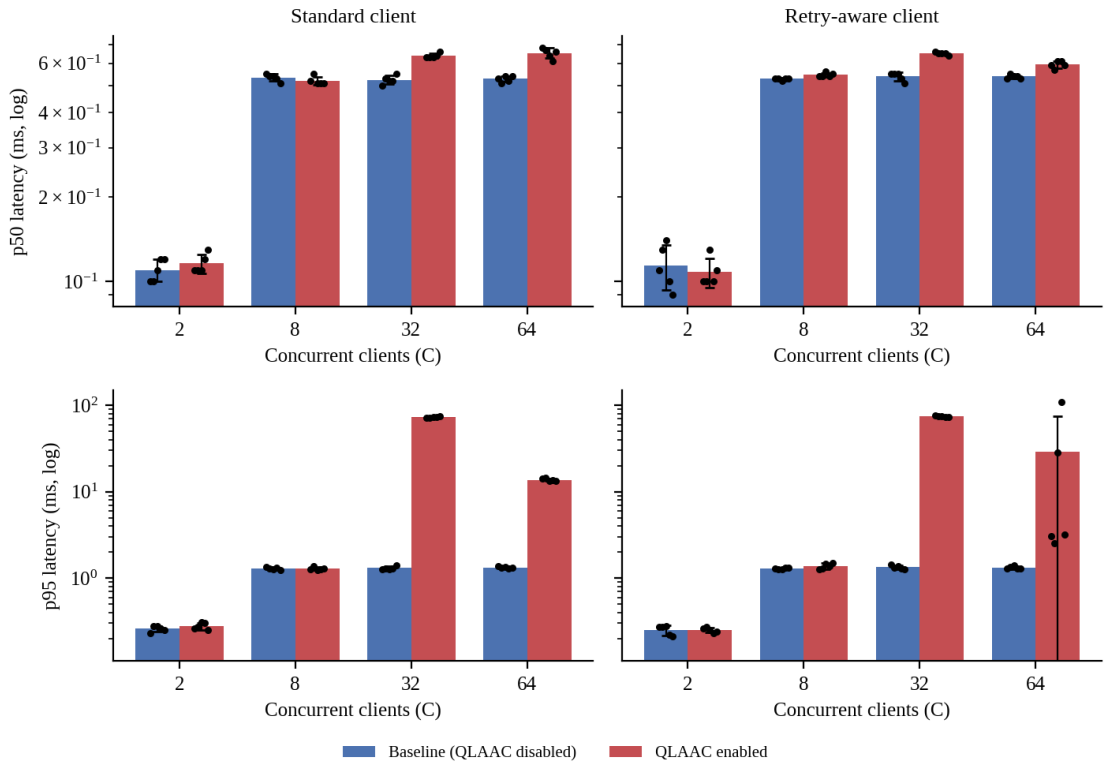



**Figure 5.3:** p50 (top) and p95 (bottom) latency of 2xx responses (log scale). At C = 32 QLAAC raises p95 about 56-fold (1.31 to 72.99 ms standard; 1.34 to 74.65 ms retry-aware) while it lowers p99.

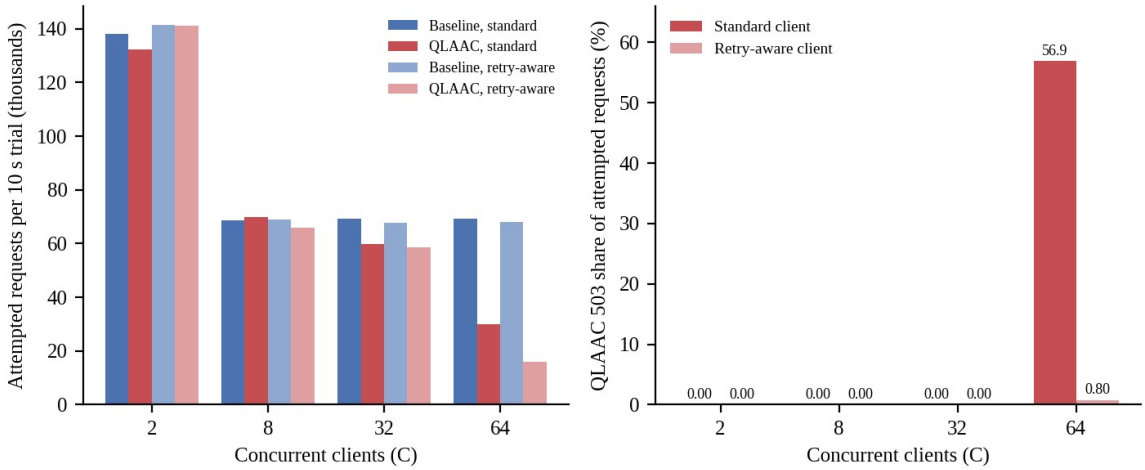



**Figure 5.4:** Left: attempted requests per 10 s trial (mean). Right: QLAAC 503 responses as a share of attempted requests. At C = 64 retry-aware clients attempt 76% fewer requests than the baseline because they wait for Retry-After.

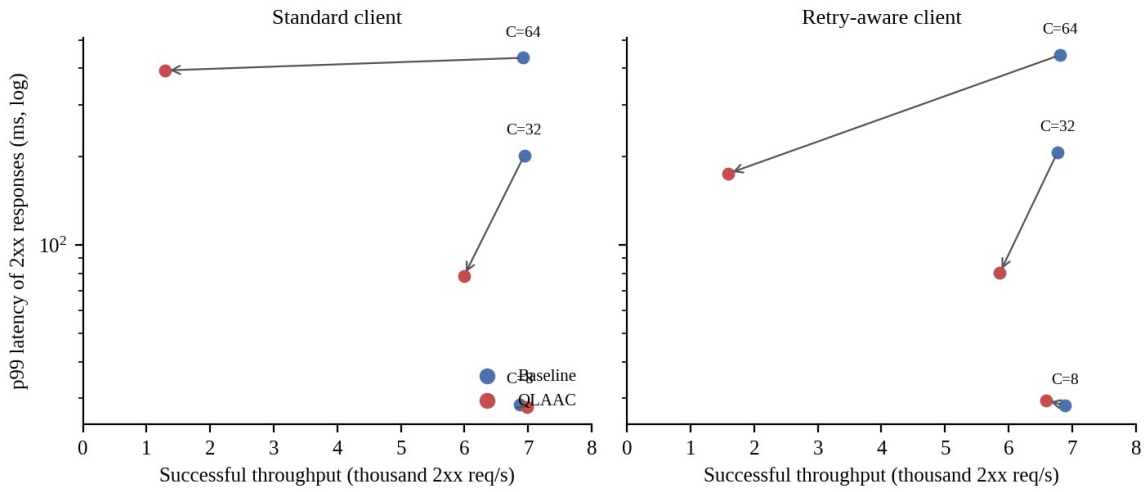



**Figure 5.5:** Throughput versus p99 trade-off. Arrows run from baseline (blue) to QLAAC (red) for C = 8, 32 and 64. At C = 8 the points nearly coincide.

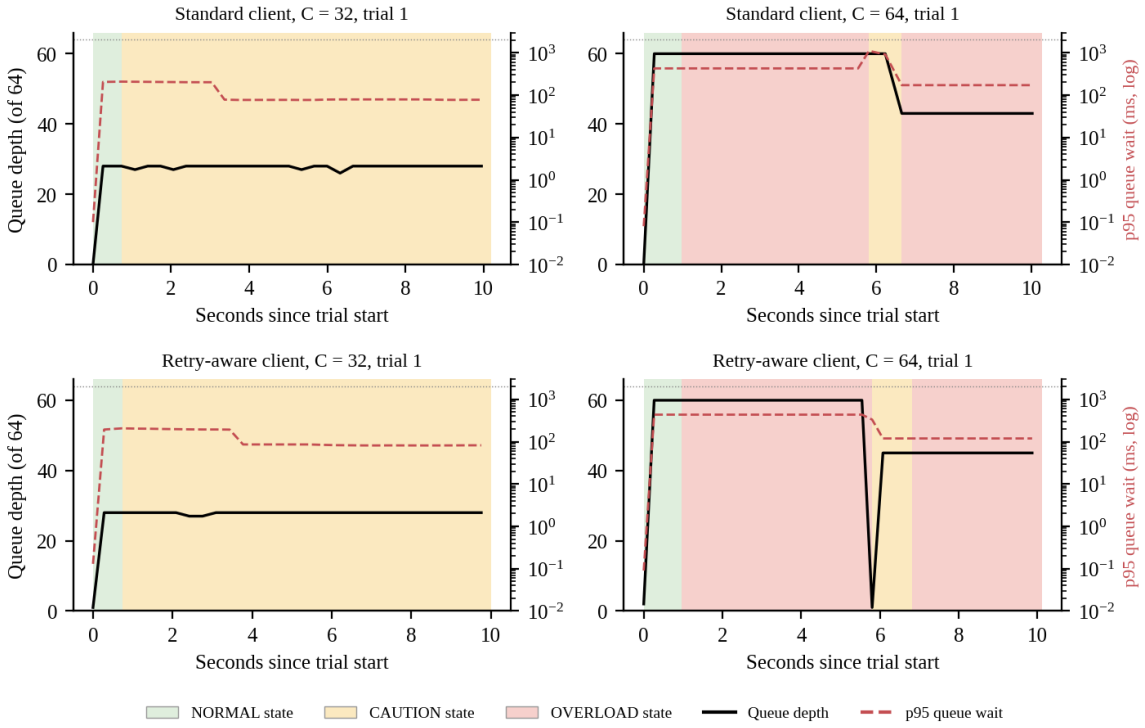



**Figure 5.6:** Controller state (shading), queue depth (black) and p95 queue wait (red, dashed, log scale) from the diagnostic timelines, trial 1 of QLAAC at C = 32 and C = 64. The first polled sample of each trial precedes the controller's first evaluation.

## 5.4 Analysis and discussion

**Run order and measurement drift.** An earlier, sequential evaluation (v3) ran all standard-mode trials first and all retry-aware trials about 8 minutes later. Baseline throughput at C = 2 differed markedly between the two blocks for identical code (21,653 against 13,149 req/s) although no 503 was ever returned, so the retry logic could not have been involved. The first trials of the two blocks started 502 s apart (CSV timestamps). This points to execution-order or

environment-related drift on the host. The raw files contain no CPU temperature or clock-frequency measurements, so thermal throttling or frequency scaling is a plausible explanation but was not demonstrated. The paired v4 design removes the confound: baseline C = 2 gives 13,806.78 ± 752.14 req/s (standard) and 14,136.06 ± 1,867.40 req/s (retry-aware), which are comparable given the scatter.

**Low and medium load (C = 2 and C = 8).** The timelines show QLAAC in NORMAL in every polled sample at these levels, so it handles requests exactly like the baseline apart from the sampler's evaluation. The measured differences are small and inconsistent in sign: throughput changes by −4.2% and −0.1% at C = 2 and by +1.7% and −4.3% at C = 8 (standard, retry-aware), and p99 at C = 8 is 28.02 against 28.54 ms and 29.49 against 28.40 ms. The trial-to-trial coefficient of variation of throughput is 4–13%, so these differences are within the run-to-run noise. QLAAC is broadly comparable to the baseline here; it neither helps nor demonstrably hurts.

**Why the baseline tail grows.** In the baseline, every client holds a connection for up to 50 requests and a worker is pinned to a connection until it ends. With C > 4, the remaining C − 4 connections wait in the queue (the baseline timelines show 28 queued at C = 32 and 60 at C = 64). The first request on a waiting connection therefore absorbs the queueing delay, while the other 49 requests on that connection are fast. Roughly 1 request in 50 (2%) is affected, which explains why the baseline p50 and p95 stay near 0.5 ms and 1.3 ms at every C ≥ 8 while p99 climbs. A simple model reproduces the p99: the waiting time is about $\lceil (C - W)/W \rceil$ service rounds, each lasting (W × 50 / X) × 1000 ms (where W = 4 workers, 50 requests per connection, and X is the measured throughput in req/s; W × 50 = 200 requests per round).

**Table 5.5:** Queue-wait model for the baseline p99 (the model uses the measured throughput, so it is a consistency check, not an independent prediction)

| Client mode | C | Throughput X (req/s) | Rounds $\lceil (C-4)/4 \rceil$ | Predicted wait = rounds × (200 / X) × 1000 ms | Observed baseline p99 (ms) |
|---|---|---|---|---|---|
| Standard | 8 | 6,869.56 | 1 | 29.1 | 28.54 |
| Standard | 32 | 6,938.70 | 7 | 201.8 | 201.93 |
| Standard | 64 | 6,922.30 | 15 | 433.4 | 434.75 |
| Retry-aware | 8 | 6,884.42 | 1 | 29.1 | 28.40 |
| Retry-aware | 32 | 6,767.70 | 7 | 206.9 | 206.97 |
| Retry-aware | 64 | 6,805.68 | 15 | 440.8 | 444.17 |

The agreement is close for both client modes, which supports the interpretation that baseline tail latency is connection-queue wait under pinned keep-alive connections. As a further check, Little's law for a closed system predicts a mean latency near C/X; for the baseline at C = 64 this

gives 9.25 ms (standard) against an observed mean of 9.27 ms, and 9.40 ms against 9.42 ms (retry-aware). The load generator and server share one host, so these absolute values include client-side overheads.

**What QLAAC does at C = 32 (CAUTION, no rejections).** In all ten QLAAC trials (five per client mode) the controller left NORMAL at about 0.7 s, entered CAUTION and stayed there; the queue peaked at 28–29 of 64 and the p95 queue wait seen by the controller reached about 195–216 ms at the start of each trial, so the queue-wait signal, not worker saturation, triggered CAUTION. No 503 was returned in any of the ten trials. With connections limited to 16 requests, each connection rotates through the workers sooner, and p99 falls by 61.3% (201.93 to 78.17 ms, standard) and 61.3% (206.97 to 80.15 ms, retry-aware). The benefit is a trade-off, not an unqualified latency reduction. p95 rises about 56-fold (1.31 to 72.99 ms in standard mode; 1.34 to 74.65 ms in retry-aware mode), p50 rises by about 21% (0.52 to 0.64 ms; 0.54 to 0.65 ms), mean latency rises by about 14–15% (4.60 to 5.28 ms; 4.72 to 5.39 ms), and successful throughput falls by 13.7% and 13.4% (6,938.70 to 5,988.46 req/s; 6,767.70 to 5,859.92 req/s). A consistency check explains the p95 jump. One request in 16 (6.25%) is now the first request on a connection and absorbs the queue wait, so the 95th percentile falls inside the waiting group, whereas in the baseline only 1 in 50 (2%) waits and p95 stays near 1.3 ms. With the same model as Table 5.5, 7 rounds of 4 × 16 requests at the measured throughput predict a wait of about 74.8 ms (standard) and 76.5 ms (retry-aware), against an observed p95/p99 of 72.99/78.17 ms and 74.65/80.15 ms. QLAAC therefore spreads a smaller delay over about three times as many requests. The throughput loss is plausibly caused by the extra connection set-up, accept and queue hops, but this cost was not isolated by a separate measurement.

**What QLAAC does at C = 64, standard client (OVERLOAD, immediate reconnect).** About 60 connections wait (queue 94% of capacity), above the 0.90 critical threshold, and the controller entered OVERLOAD at about 0.95 s in every trial. The listener then answers new connections with 503; because standard clients reconnect at once, the five trials recorded a mean of 17,126.8 HTTP 503 rejections per trial (sample SD 171.3) and a mean of 30,079.4 attempted requests per trial (sample SD 532.4), giving a rejection rate of 56.94% using the ratio of means. Successful throughput fell by 81.3% (6,922.30 to 1,295.26 req/s). p99 improved only 9.4% (434.75 to 393.73 ms) and p95 rose from 1.33 to 13.74 ms. In this mode QLAAC did not achieve a worthwhile latency benefit for its throughput cost and is classed as unsuccessful. The timelines show recovery to CAUTION after about five seconds and renewed OVERLOAD in

three of five trials, consistent with the slow recovery rule (five windows). It must also be noted that the baseline queue never exceeded 60 of 64 slots and rejected nothing, so these data do not show that shedding was needed to prevent queue exhaustion in this workload.

**What QLAAC does at C = 64, retry-aware client.** The same OVERLOAD episode occurs, but each trial records exactly 128 rejections, a rejection rate of 0.80% of attempts. Clients that honour Retry-After: 5 sleep, so the offered load collapses: attempted requests fall by 76.4% (68,056.8 to 16,042.8 per trial) and the clients accumulate 511.91 client-thread-seconds of back-off of a possible 640. Successful throughput falls by 76.6% (6,805.68 to 1,591.48 req/s) while p99 falls by 60.6% (444.17 to 175.15 ms). This is a trade-off, not a throughput improvement: the low rejection rate hides the fact that most client time is spent waiting, and part of the throughput loss is a consequence of the fixed 10 s window in which a 5 s Retry-After consumes up to half of the trial. Longer runs, in which clients resume and stagger their retries, were not tested. The results also vary strongly between trials: throughput ranges from 1,197 to 2,421 req/s (CV 33%), p95 from 2.55 to 108.27 ms and p99 from 121 to 225 ms (175.15 ± 47.31 ms), so five trials cannot characterise this regime precisely.

**Table 5.6:** Summary of findings by scenario (our own reading of Tables 5.3 and 5.4; not a statistical classification)

| Scenario | Client mode | Finding |
|---|---|---|
| C = 2 | both | Comparable to baseline (throughput −4.2% / −0.1%); QLAAC stays in NORMAL |
| C = 8 | both | Comparable (throughput +1.7% / −4.3%; p99 within about 4%); QLAAC stays in NORMAL |
| C = 32 | both | Trade-off: p99 −61.3%, no rejections; but throughput −13.7% / −13.4%, p95 about 56× higher, p50 about +21% |
| C = 64 | standard | Unsuccessful: throughput −81.3%, 56.9% of attempts rejected, p99 only −9.4% |
| C = 64 | retry-aware | Trade-off: p99 −60.6%, rejection rate 0.80%, but throughput −76.6% because clients back off; high trial-to-trial variance |

## 5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails

**Strengths.** The controller is small (299 lines including comments), uses only in-process signals, is unit-tested for transitions, hysteresis and the moderate-queue case, and in the measured trials it did not intervene at C = 2 and C = 8 (NORMAL in every polled sample, results comparable to the baseline). At C = 32 it reduced p99 by about 61% without rejecting a single request. The benchmark accounting is strict about what counts as success and separates

response latency from client back-off time, which makes the cost of protection visible rather than hidden.

**Weaknesses and verified limitations.**

- **Cost at moderate overload.** At C = 32 successful throughput falls by about 13–14%, p50 and mean latency rise by about 21% and 14–15%, and p95 rises about 56-fold; only the extreme tail improves.
- **Lagging queue-wait signal.** A queue wait is recorded only when a worker dequeues the connection, so connections that are still waiting contribute no sample, and the p95 covers the last 5 s sampled once per second. The controller reacts after the delay has been experienced (the p95 queue wait was already about 200 ms when CAUTION began at C = 32).
- **Unstable behaviour at overload.** In retry-aware mode at C = 64 throughput varies by 33% (CV) and p95 between 2.55 and 108.27 ms across five trials; in both modes OVERLOAD can re-trigger after a brief CAUTION period.
- **Client-dependent outcome.** With immediate-reconnect clients, shedding creates a reconnect storm (about 17,000 rejections per trial) and a severe throughput collapse. The benefit under retry-aware clients depends on clients actually honouring Retry-After, which the project could only test with its own harness.
- **Coarse, untuned control.** One-second sampling, five-window recovery, the 16-request cap and all thresholds were not systematically tuned, and no sensitivity study was done. The recovery rule keeps OVERLOAD active for at least five windows after pressure subsides.
- **Historical counter inconsistency.** In an earlier pre-audit run, qlaac_shed_requests overcounted relative to 503s sent because workers evaluated admission state directly (resolved in current source; Section 4.4). The counters track distinct events (QLAAC shed decisions, all connection rejections, and successful server-side 503 writes), so neither of the first and last is guaranteed to exceed the other.

**Measurement limitations.** One workload (a tiny JSON health endpoint, no disk I/O or large bodies); one host with client and server sharing CPU time on the loopback interface; four concurrency levels; five trials per cell; a Python threaded client, which may itself limit offered load; fixed 10 s trials, which distort retry-aware results because of the 5 s Retry-After; no recorded hardware details, temperature or clock frequency for the v4 runs; and controller state

sampled by a diagnostic poller at about 0.25 s whose last sample may precede the end of a trial. Fresh server processes remove controller-state carry-over, but a 1 s warm-up at C = 2 precedes every timed trial.

**Conditions under which QLAAC fails to improve performance or reduces successful throughput.** (i) When concurrency does not exceed the worker count or the queue stays short (C = 2 and 8): no clear difference. (ii) Whenever the objective is throughput, median, mean or p95 latency rather than p99: at C = 32 QLAAC loses 13–14% throughput and raises p95 about 56-fold. (iii) When most load comes from clients that retry immediately: at C = 64 throughput falls by 81% and 57% of attempts are rejected for a 9% p99 gain. (iv) When clients honour Retry-After over a short window: p99 improves but successful throughput falls by 77% as clients wait. (v) When a stable overload response is required: C = 64 retry-aware results vary widely between trials.

# 6. CONCLUSION AND FUTURE WORK

## 6.1 Summary of work and key findings

A multi-threaded HTTP/1.1 server was implemented in C17 on raw TCP sockets, comprising a listener, bounded connection queue, four-worker pool, streaming parser for a documented subset of RFC 9112, router, static-file delivery with traversal checks, metrics with queue-wait telemetry and a dashboard. The current recorded automated suite contains 92 tests (44 C unit tests and 48 Python tests); the run documented in Appendix B.8 reports all 92 passing. The earlier 80-test run is retained as a dated historical record in Appendix B.2. QLAAC, a three-state queue-aware admission controller with keep-alive fair sharing and hysteresis, was compared against the same server with the controller disabled, using a paired benchmark with fresh server processes that separates 2xx responses from every other outcome. The quantitative findings (five 10-second trials per point; mean ± sample SD; standard / retry-aware client) are:

- C = 2 and C = 8: comparable to the baseline (C = 2: 13,806.78 ± 752.14 vs 13,230.22 ± 566.31 req/s and 14,136.06 ± 1,867.40 vs 14,115.42 ± 856.21 req/s; C = 8: 6,869.56 vs 6,989.02 and 6,884.42 vs 6,587.70 req/s). QLAAC remained in NORMAL.
- C = 32: p99 201.93 → 78.17 ms and 206.97 → 80.15 ms (both −61.3%), no 503 responses; but throughput 6,938.70 → 5,988.46 and 6,767.70 → 5,859.92 req/s (−13.7% and −13.4%) and p95 1.31 → 72.99 ms and 1.34 → 74.65 ms.
- C = 64, standard clients: throughput 6,922.30 → 1,295.26 req/s (−81.3%), 56.9% of attempts rejected with 503, p99 only 434.75 → 393.73 ms (−9.4%).
- C = 64, retry-aware clients: p99 444.17 → 175.15 ms (−60.6%), 128 rejections per trial (0.80% of attempts), but throughput 6,805.68 → 1,591.48 req/s (−76.6%) because clients waited (511.91 client-thread-seconds of back-off per trial) and offered 76% fewer requests.
- Baseline tail latency is explained by connection-queue wait behind pinned keep-alive connections (model agreement in Table 5.5).

QLAAC therefore trades successful throughput, p50, p95 and mean latency (at C = 32) or most of the throughput (at C = 64) for lower p99 latency of admitted requests. It is a trade-off whose value depends on the objective and on client retry behaviour, and it should not be described as universally better.

### 6.2 Limitations

Implementation: HTTP/1.1 support is a documented subset. Chunked request decoding is not implemented and requests with Transfer-Encoding: chunked receive 501, an explicit deviation from the full RFC 9112 §6.1/§7.1 requirement; TLS and range requests are also out of scope. The parser rejects bare LF as a defensive policy even though RFC 9112 §2.2 permits a recipient to recognize a single LF as a line terminator. Blocking worker threads limit connection scalability and can be occupied by slow clients (Slowloris-style starvation). Measurement: single host and workload, Python load generator, closed-loop clients, five trials per point, fixed 10 s trials, no recorded hardware details, temperature or clock frequency for the historical v4 runs, and only a polled approximation of controller state over time. No significance tests were run. Scalability beyond 64 clients, other workloads (large files, POST bodies) and other thread/queue sizes were not evaluated. The 16-request cap and thresholds were not tuned. Security evaluation was limited to a targeted internal audit and 12 regression tests, plus sanitizer builds of the C unit tests; no external penetration testing or full RFC compliance testing was performed.

### 6.3 Future enhancements

The following are proposals, not completed work:

- Make the queue-wait signal leading rather than lagging, for example by also using the age of the oldest connection still in the queue.
- Study threshold and fair-sharing-cap sensitivity systematically (for example 8, 16, 32 requests), and test whether a cap that adapts to queue depth reduces the C = 32 throughput cost.
- Add an open-loop load generator (for example wrk, or a fixed-rate client) with Retry-After-aware clients, longer trials that let backed-off clients resume, and the recording of controller state, queue depth and shed counts as proper time series, including recovery time.
- Record host metadata, CPU temperature and clock frequency during benchmarks to test whether run-order drift is thermal or frequency related, and evaluate other workloads (static files of several sizes, POST echo) and worker/queue configurations with more trials and confidence intervals.
- Extend protocol coverage to chunked bodies, range requests and TLS, and evaluate event-driven I/O to reduce slow-client worker starvation.

# 7. REFERENCES

***References (IEEE style)***


[1] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP/1.1," Internet Engineering Task Force, RFC 9112 (STD 99), Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9112.html

[2] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP Semantics," Internet Engineering Task Force, RFC 9110 (STD 97), Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[3] W. Eddy, Ed., "Transmission Control Protocol (TCP)," Internet Engineering Task Force, RFC 9293 (STD 7), Aug. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293.html

[4] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in Proc. 18th ACM Symp. Operating Systems Principles (SOSP), Banff, AB, Canada, 2001, pp. 230–243. doi: 10.1145/502059.502057.

[5] M. Welsh and D. Culler, "Adaptive overload control for busy Internet servers," in Proc. 4th USENIX Symp. Internet Technologies and Systems (USITS), Seattle, WA, USA, Mar. 2003, pp. 43–57. [Online]. Available: https://www.usenix.org/conference/usits-03/adaptive-overload-control-busy-internet-servers

[6] B. Schroeder and M. Harchol-Balter, "Web servers under overload: How scheduling can help," ACM Trans. Internet Technol., vol. 6, no. 1, pp. 20–52, Feb. 2006. doi: 10.1145/1125274.1125276.

[7] D. Pariag, T. Brecht, A. S. Harji, P. A. Buhr, and A. Shukla, "Comparing the performance of web server architectures," in Proc. 2nd ACM SIGOPS/EuroSys European Conf. Computer Systems (EuroSys), Lisbon, Portugal, 2007, pp. 231–243. doi: 10.1145/1272998.1273021.

[8] K. Nichols and V. Jacobson, "Controlling queue delay," ACM Queue, vol. 10, no. 5, pp. 20–34, May 2012. doi: 10.1145/2208917.2209336.

[9] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," ACM Queue, vol. 9, no. 11, pp. 40–54, Nov. 2011. doi: 10.1145/2063166.2071893.

[10] J. Dean and L. A. Barroso, "The tail at scale," Commun. ACM, vol. 56, no. 2, pp. 74–80, Feb. 2013. doi: 10.1145/2408776.2408794.

[11] J. D. C. Little, "A proof for the queuing formula: L = λW," Oper. Res., vol. 9, no. 3, pp. 383–387, Jun. 1961. doi: 10.1287/opre.9.3.383.

# APPENDIX A: IMPORTANT SOURCE CODE OR CONFIGURATION LISTINGS

Excerpts are shortened for the report; line comments are condensed. The full code is in the supplied repository.

### A.1 QLAAC overload and caution conditions (src/adaptive_controller.c)

This is the core decision logic. Worker saturation alone is not an overload condition and, with a short queue and healthy latency, not even a caution condition. Caution and overload require queue, queue-wait or latency evidence; critical queue wait raises OVERLOAD only when the queue is also growing or the controller is already in OVERLOAD.

```
bool has_queue_presence = (queue_occupancy_ratio >= 0.20);
bool queue_wait_elevated = (cfg->queue_wait_caution_ms > 0.0 && p95_queue_wait_ms > 0.0 &&
        p95_queue_wait_ms >= cfg->queue_wait_caution_ms && has_queue_presence);
bool queue_wait_critical = (... >= cfg->queue_wait_overload_ms && has_queue_presence);
bool sustained_pressure_candidate =
    (worker_saturated && queue_occupancy_ratio >= 0.50) ||
    (queue_growing && (queue_wait_elevated || latency_elevated));
bool queue_wait_overload = queue_wait_critical &&
    (queue_growing || (controller->current_state == QLAAC_STATE_OVERLOAD));
bool is_overload_condition = queue_critical || latency_critical ||
    queue_wait_overload || (worker_saturated && queue_growing) ||
    sustained_pressure_overload;
bool worker_stress_with_queue = (worker_caution || worker_saturated) &&
    (queue_occupancy_ratio >= 0.50 || queue_wait_elevated || latency_elevated);
bool is_caution_condition = is_overload_condition || queue_growing ||
    queue_wait_elevated || queue_wait_critical || latency_elevated ||
    worker_stress_with_queue;
```

### A.2 Accept-time admission and 503 shedding (src/server.c)

The listener asks the controller for a decision for every accepted connection; in OVERLOAD it answers 503 itself, and otherwise attempts a non-blocking push to the bounded queue (a full queue also yields a 503 with Retry-After: 5).

```
QlaacDecision decision = adaptive_controller_evaluate(&ctx->controller);
if (decision == QLAAC_ACTION_REJECT_OVERLOAD) {
    metrics_record_connection_rejected(&ctx->metrics);
    const char *msg_503 = "HTTP/1.1 503 Service Unavailable\r\n"
        "Content-Type: text/html; charset=utf-8\r\nContent-Length: 96\r\n"
        "Connection: close\r\nRetry-After: 5\r\n\r\n<html>...</html>\n";
    ... tcp_send_all(client.fd, msg_503, strlen(msg_503), deadline, &sent_bytes);
    tcp_lingering_close(&client.fd, 50);
    continue;
}
int push_rc = connection_queue_push(&ctx->queue, &client, 0);   /* non-blocking */
if (push_rc != 0) { /* queue full: 503 with Retry-After: 5 (Content-Length: 98) */ }
```

### A.3 CAUTION keep-alive throttle in the worker loop (src/thread_pool.c)

This is keep-alive fair sharing. In CAUTION a connection is limited to at most 16 requests with a 2 s idle timeout; in OVERLOAD to 16 requests with a 1 s idle timeout. Worker threads query the controller using adaptive_controller_get_state(), a pure, idempotent query that triggers no

evaluation, state transition or shed-counter increment. Admission rejection and shed accounting occur solely at the listener's accept gate in src/server.c.

```
QlaacState qstate = adaptive_controller_get_state(&ctx->controller);
if (adaptive_control_enabled && qstate == QLAAC_STATE_OVERLOAD) {
    keepalive_timeout_sec = 1;
    max_requests = (keepalive_max_requests > 16) ? 16 : keepalive_max_requests;
} else if (adaptive_control_enabled && qstate == QLAAC_STATE_CAUTION) {
    keepalive_timeout_sec = 2;
    max_requests = (keepalive_max_requests > 16) ? 16 : keepalive_max_requests;
} else { /* NORMAL: configured values (100 requests, 5 s) */ }
```

### A.4 Configuration (config/server.conf.example, QLAAC section)

Threshold values used in Table 3.1; adaptive_control_enabled is false in the example file, which is why the baseline is the default.

```
worker_threads = 4
connection_queue_capacity = 64
keepalive_max_requests = 100
keepalive_timeout_sec = 5
adaptive_control_enabled = false
sampling_window_ms = 1000
queue_caution_threshold = 0.70
queue_overload_threshold = 0.90
latency_caution_ms = 50.0
latency_overload_ms = 150.0
queue_wait_caution_ms = 25.0
queue_wait_overload_ms = 100.0
worker_caution_threshold = 0.85
worker_overload_threshold = 1.00
sustained_pressure_threshold = 3
recovery_sample_count = 5
```

### A.5 Harness accounting (benchmark/run_experiment.py)

The harness counts a response as a success only when the parsed status code is 2xx, and computes throughput and latency from those responses only. In retry-aware mode a 5xx response makes the client wait for Retry-After seconds (1 s if the header is missing or invalid); the waiting time is accumulated separately and is not part of any latency sample.

```
if 200 <= status_code < 300:
    local_2xx += 1
    local_latencies.append(latency_ms)
elif 300 <= status_code < 400:  local_3xx += 1
elif 400 <= status_code < 500:  local_4xx += 1
elif 500 <= status_code < 600:
    local_5xx += 1
    if retry_aware:                       # honour Retry-After
        backoff_dur = float(ra_str) if valid else 1.0
        stop_event.wait(timeout=backoff_dur)
        local_backoff_sec += elapsed_wait  # -> total_backoff_sec
...
successful_rps = success_2xx / duration_sec
p99 = success_latencies[int(0.99 * (count - 1))]      # N/A if no 2xx
```

### A.6 Commands

Build and test (start the server first so that the Python tests do not skip; scripts/run_all_tests.sh does this itself; invoke it with bash because the archive did not preserve its executable bit):

```
  make                      # builds bin/server
  make test                 # builds; runs the 5 C test binaries (44 tests) + Python tests
  bash scripts/run_all_tests.sh   # C tests + Python regression suite (counts in B.2 and B.8)
python3 -m unittest tests/test_readiness_check.py -v   # run from the current checkout;
see Appendix B.8
  ./bin/server -c config/server.conf.example -p 8080          # baseline
  ./bin/server -c config/server.conf.example -p 8080 -a       # QLAAC enabled
  python3 benchmark/run_reproducible_evaluation.py --client-mode standard   # paired runner
  python3 benchmark/run_reproducible_evaluation.py --client-mode retry-aware
  python3 benchmark/verify_benchmark_statistics.py --v4-standard   # recompute
  python3 benchmark/verify_benchmark_statistics.py --v4-retryaware
```

# APPENDIX B: ADDITIONAL SCREENSHOTS, PACKET CAPTURES, RAW RESULT TABLES

## B.1 Application screenshots (captured from the running server)

These captures were taken from the live dashboard (supplied files 02, 03, 04 and 05). They show functional behaviour, not benchmark results, and were captured before the queue-wait display was added to public/index.html, so the current dashboard layout differs. The capture in Figure B.4 shows the controller panel in the NORMAL state at idle, not under overload.

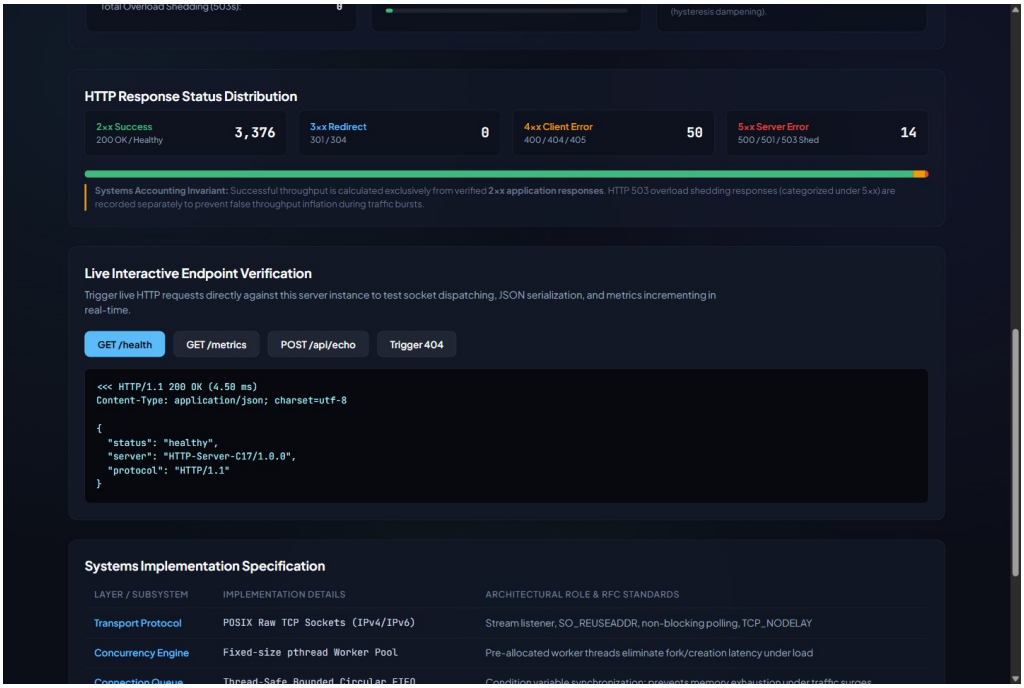



**Figure B.1:** API console: GET /health returning HTTP/1.1 200 OK with the JSON health payload (4.50 ms round trip as displayed), and the status-class distribution (2xx 3,376; 4xx 50; 5xx 14).

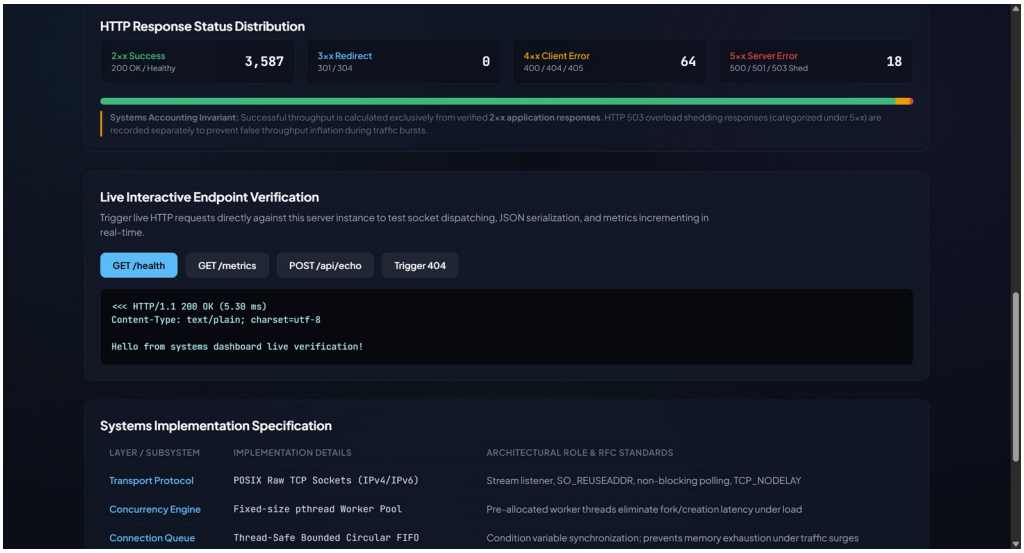



**Figure B.2:** API console: POST /api/echo returning HTTP/1.1 200 OK with the request body echoed (5.30 ms as displayed).

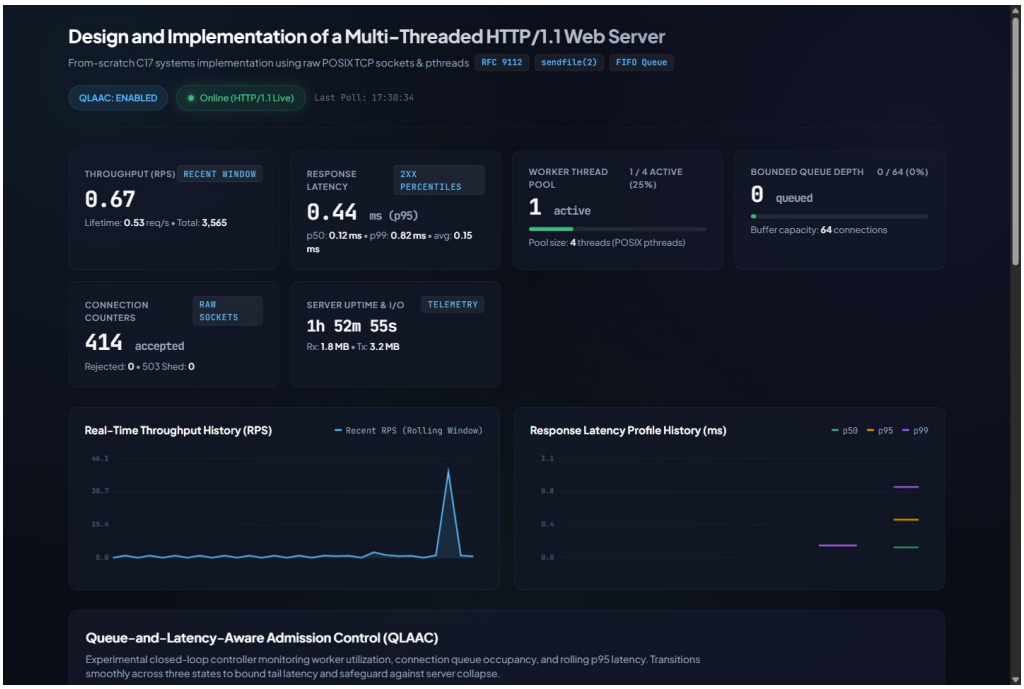



**Figure B.3:** Dashboard after a short burst of requests: throughput spike, 2xx percentiles p50 0.12 ms, p95 0.44 ms, p99 0.82 ms (local manual burst; not part of the benchmark).

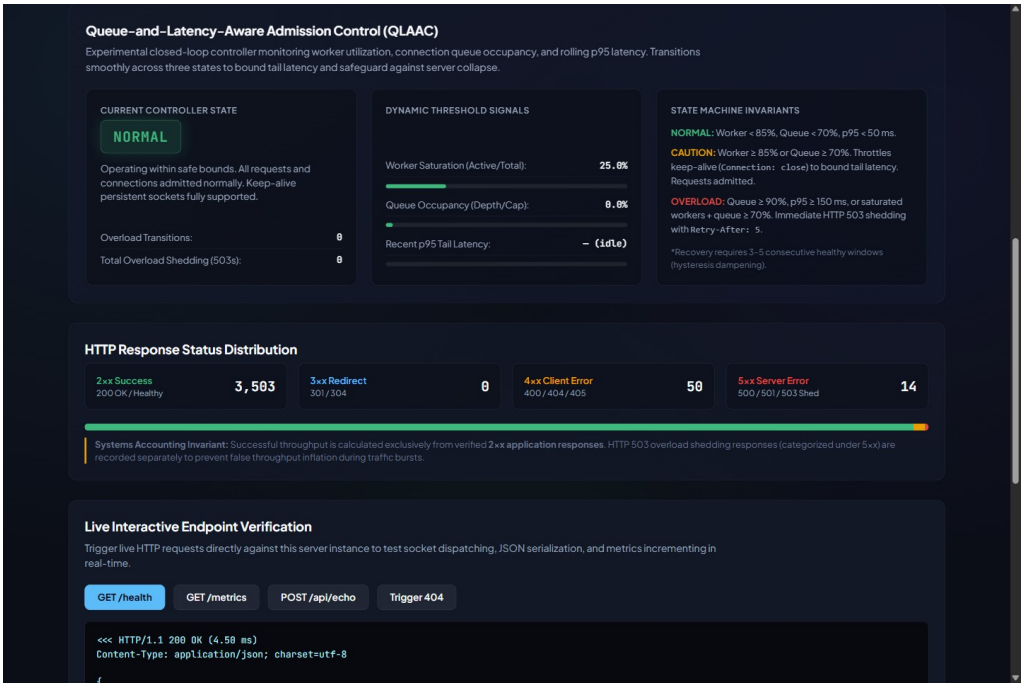



**Figure B.4:** QLAAC controller panel (state NORMAL, worker saturation 25%, queue 0%, zero shed) with the displayed state-machine invariants.

## B.2 Test execution output

The supplied image 06_test_results.png is omitted because it could not be verified as an authentic terminal capture. The test output below records the relevant command results from the source-package verification run in a Linux sandbox (Ubuntu 24.04, GCC 13.3.0, Python 3.12.3, 1 logical CPU). Compiler command lines and QLAAC log lines are omitted.

```
$ bash scripts/run_all_tests.sh
bin/test_controller         [PASS] x13   (--- All QLAAC Controller Tests Passed ---)
bin/test_parser             [PASS] x14   (--- All HTTP Parser Tests Passed ---)
bin/test_queue              [PASS] x4    (--- All Connection Queue Tests Passed ---)
bin/test_responses          [PASS] x5    (--- All HTTP Response Tests Passed ---)
bin/test_server_metrics     [PASS] x8    (--- All Server Metrics Tests Passed ---)
                                         -> 44 / 44 C unit tests passed
(server started on port 8080 by the script)
python3 -m unittest discover -s tests -p "test_*.py" -v
Ran 36 tests in 0.518s
OK                                       -> 36 / 36 passed, 0 skipped, exit status 0

(same command with NO server listening:  Ran 36 tests ... OK (skipped=21))
```

Total: 80 of 80 tests pass when a server is running (44 C unit tests and 36 Python tests). Additional checks run in this revision: the five C unit-test executables were rebuilt and run under AddressSanitizer + UndefinedBehaviorSanitizer (44 / 44 passed, no sanitizer report) and under ThreadSanitizer (44 / 44 passed, no report); gcc -fanalyzer with the Makefile flags over the 12 files in src/ produced 0 warnings. The sanitizer runs cover the unit-test harnesses only, not a long-running server process and not the Python tests. The earlier bounded-shutdown integration test (test_graceful_shutdown_drains_idle_keepalive) is part of the 36 Python tests. This is the earlier 80-

test suite; the 12 additional readiness-check tests are recorded separately in Appendix B.8, where the current recorded total is 92.

**Final source-package recheck and complete performance rerun (21 October 2026):** At that stage, the clean build and all 80 automated tests passed (44 C, 36 Python). The complete paired benchmark finished on port 18080 after health/metrics checks because port 8080 was occupied. All 80 timed trials and CSV accounting checks passed. The host differs from historical v4, so Chapter 5 retains the archived results and fresh measurements are separated in Appendix B.7. This dated run predates the readiness-check follow-up in B.8; the performance benchmark was not rerun after that code refinement. Source code and archived v4 data hashes were unchanged for this recorded run.

### B.3 Packet captures

No packet captures (pcap files or Wireshark screenshots) were included in the supplied materials, and none are presented in this report.

### B.4 Raw result tables

Per-trial values from the four v4 paired CSV files used in Chapter 5 (attempted requests, 2xx, 503s, successful throughput and latency percentiles of 2xx responses). All other columns (3xx, 4xx, timeouts, transport errors, invalid responses) are zero in every trial.

**Table B.1:** Raw per-trial data, standard client (experiment_{baseline,qlaac} _standard_v4_paired_20261019.csv); latencies in ms

| C | Trial | Policy | Attempted | 2xx | 503 | 2xx req/s | p50 | p95 | p99 |
|---|---|---|---|---|---|---|---|---|---|
| 2 | 1 | Baseline | 150,517 | 150,517 | 0 | 15,051.70 | 0.10 | 0.23 | 0.41 |
| 2 | 1 | QLAAC | 140,320 | 140,320 | 0 | 14,032.00 | 0.11 | 0.26 | 0.45 |
| 2 | 2 | Baseline | 139,822 | 139,822 | 0 | 13,982.20 | 0.10 | 0.28 | 0.46 |
| 2 | 2 | QLAAC | 135,553 | 135,553 | 0 | 13,555.30 | 0.11 | 0.27 | 0.47 |
| 2 | 3 | Baseline | 133,956 | 133,956 | 0 | 13,395.60 | 0.11 | 0.28 | 0.47 |
| 2 | 3 | QLAAC | 127,548 | 127,548 | 0 | 12,754.80 | 0.11 | 0.31 | 0.53 |
| 2 | 4 | Baseline | 132,575 | 132,575 | 0 | 13,257.50 | 0.12 | 0.26 | 0.42 |
| 2 | 4 | QLAAC | 126,853 | 126,853 | 0 | 12,685.30 | 0.12 | 0.30 | 0.49 |
| 2 | 5 | Baseline | 133,469 | 133,469 | 0 | 13,346.90 | 0.12 | 0.25 | 0.40 |
| 2 | 5 | QLAAC | 131,237 | 131,237 | 0 | 13,123.70 | 0.13 | 0.25 | 0.38 |
| 8 | 1 | Baseline | 66,295 | 66,295 | 0 | 6,629.50 | 0.55 | 1.36 | 29.53 |
| 8 | 1 | QLAAC | 69,799 | 69,799 | 0 | 6,979.90 | 0.52 | 1.27 | 28.06 |
| 8 | 2 | Baseline | 68,370 | 68,370 | 0 | 6,837.00 | 0.54 | 1.29 | 28.61 |
| 8 | 2 | QLAAC | 65,330 | 65,330 | 0 | 6,533.00 | 0.55 | 1.38 | 29.71 |
| 8 | 3 | Baseline | 68,924 | 68,924 | 0 | 6,892.40 | 0.54 | 1.26 | 28.57 |
| 8 | 3 | QLAAC | 72,180 | 72,180 | 0 | 7,218.00 | 0.51 | 1.23 | 27.13 |
| 8 | 4 | Baseline | 68,147 | 68,147 | 0 | 6,814.70 | 0.53 | 1.33 | 28.48 |
| 8 | 4 | QLAAC | 71,481 | 71,481 | 0 | 7,148.10 | 0.51 | 1.26 | 27.46 |
| 8 | 5 | Baseline | 71,742 | 71,742 | 0 | 7,174.20 | 0.51 | 1.25 | 27.49 |
| 8 | 5 | QLAAC | 70,661 | 70,661 | 0 | 7,066.10 | 0.51 | 1.29 | 27.74 |
| 32 | 1 | Baseline | 72,563 | 72,563 | 0 | 7,256.30 | 0.50 | 1.27 | 193.42 |

| C | Trial | Policy | Attempted | 2xx | 503 | 2xx req/s | p50 | p95 | p99 |
|---|---|---|---|---|---|---|---|---|---|
| 32 | 1 | QLAAC | 60,782 | 60,782 | 0 | 6,078.20 | 0.63 | 71.71 | 76.60 |
| 32 | 2 | Baseline | 69,070 | 69,070 | 0 | 6,907.00 | 0.53 | 1.29 | 203.12 |
| 32 | 2 | QLAAC | 60,579 | 60,579 | 0 | 6,057.90 | 0.63 | 72.48 | 77.78 |
| 32 | 3 | Baseline | 70,088 | 70,088 | 0 | 7,008.80 | 0.52 | 1.27 | 200.05 |
| 32 | 3 | QLAAC | 59,807 | 59,807 | 0 | 5,980.70 | 0.63 | 73.08 | 78.68 |
| 32 | 4 | Baseline | 69,596 | 69,596 | 0 | 6,959.60 | 0.52 | 1.30 | 201.75 |
| 32 | 4 | QLAAC | 60,047 | 60,047 | 0 | 6,004.70 | 0.64 | 72.63 | 77.94 |
| 32 | 5 | Baseline | 65,618 | 65,618 | 0 | 6,561.80 | 0.55 | 1.42 | 211.32 |
| 32 | 5 | QLAAC | 58,208 | 58,208 | 0 | 5,820.80 | 0.66 | 75.05 | 79.83 |
| 64 | 1 | Baseline | 68,630 | 68,630 | 0 | 6,863.00 | 0.53 | 1.37 | 447.83 |
| 64 | 1 | QLAAC | 29,676 | 12,604 | 17,072 | 1,260.40 | 0.68 | 14.20 | 416.72 |
| 64 | 2 | Baseline | 71,083 | 71,083 | 0 | 7,108.30 | 0.51 | 1.31 | 426.34 |
| 64 | 2 | QLAAC | 29,523 | 12,550 | 16,973 | 1,255.00 | 0.67 | 14.39 | 409.79 |
| 64 | 3 | Baseline | 67,754 | 67,754 | 0 | 6,775.40 | 0.54 | 1.36 | 437.74 |
| 64 | 3 | QLAAC | 30,494 | 13,074 | 17,420 | 1,307.40 | 0.64 | 13.24 | 380.79 |
| 64 | 4 | Baseline | 70,566 | 70,566 | 0 | 7,056.60 | 0.52 | 1.29 | 424.82 |
| 64 | 4 | QLAAC | 30,765 | 13,657 | 17,108 | 1,365.70 | 0.61 | 13.51 | 363.39 |
| 64 | 5 | Baseline | 68,082 | 68,082 | 0 | 6,808.20 | 0.54 | 1.33 | 437.02 |
| 64 | 5 | QLAAC | 29,939 | 12,878 | 17,061 | 1,287.80 | 0.66 | 13.38 | 397.98 |

**Table B.2:** Raw per-trial data, retry-aware client (experiment_{baseline,qlaac}_retry-aware_v4_paired_20261019.csv); latencies in ms; backoff in client-thread-seconds

| C | Trial | Policy | Attempted | 2xx | 503 | 2xx req/s | p50 | p95 | p99 | Backoff (s) |
|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 1 | Baseline | 135,590 | 135,590 | 0 | 13,559.00 | 0.11 | 0.27 | 0.44 | 0.00 |
| 2 | 1 | QLAAC | 140,740 | 140,740 | 0 | 14,074.00 | 0.10 | 0.26 | 0.46 | 0.00 |
| 2 | 2 | Baseline | 129,769 | 129,769 | 0 | 12,976.90 | 0.13 | 0.27 | 0.44 | 0.00 |
| 2 | 2 | QLAAC | 143,676 | 143,676 | 0 | 14,367.60 | 0.10 | 0.27 | 0.48 | 0.00 |
| 2 | 3 | Baseline | 119,956 | 119,956 | 0 | 11,995.60 | 0.14 | 0.28 | 0.43 | 0.00 |
| 2 | 3 | QLAAC | 128,825 | 128,825 | 0 | 12,882.50 | 0.13 | 0.25 | 0.38 | 0.00 |
| 2 | 4 | Baseline | 157,776 | 157,776 | 0 | 15,777.60 | 0.10 | 0.22 | 0.37 | 0.00 |
| 2 | 4 | QLAAC | 152,686 | 152,686 | 0 | 15,268.60 | 0.10 | 0.23 | 0.39 | 0.00 |
| 2 | 5 | Baseline | 163,712 | 163,712 | 0 | 16,371.20 | 0.09 | 0.21 | 0.35 | 0.00 |
| 2 | 5 | QLAAC | 139,844 | 139,844 | 0 | 13,984.40 | 0.11 | 0.24 | 0.40 | 0.00 |
| 8 | 1 | Baseline | 69,009 | 69,009 | 0 | 6,900.90 | 0.53 | 1.30 | 28.38 | 0.00 |
| 8 | 1 | QLAAC | 68,341 | 68,341 | 0 | 6,834.10 | 0.54 | 1.26 | 28.61 | 0.00 |
| 8 | 2 | Baseline | 69,078 | 69,078 | 0 | 6,907.80 | 0.53 | 1.27 | 28.44 | 0.00 |
| 8 | 2 | QLAAC | 67,364 | 67,364 | 0 | 6,736.40 | 0.54 | 1.30 | 28.95 | 0.00 |
| 8 | 3 | Baseline | 69,956 | 69,956 | 0 | 6,995.60 | 0.52 | 1.26 | 28.05 | 0.00 |
| 8 | 3 | QLAAC | 63,751 | 63,751 | 0 | 6,375.10 | 0.56 | 1.48 | 30.32 | 0.00 |
| 8 | 4 | Baseline | 68,043 | 68,043 | 0 | 6,804.30 | 0.53 | 1.32 | 28.50 | 0.00 |
| 8 | 4 | QLAAC | 66,520 | 66,520 | 0 | 6,652.00 | 0.54 | 1.38 | 29.14 | 0.00 |
| 8 | 5 | Baseline | 68,135 | 68,135 | 0 | 6,813.50 | 0.53 | 1.32 | 28.63 | 0.00 |
| 8 | 5 | QLAAC | 63,409 | 63,409 | 0 | 6,340.90 | 0.55 | 1.51 | 30.42 | 0.00 |
| 32 | 1 | Baseline | 65,208 | 65,208 | 0 | 6,520.80 | 0.55 | 1.45 | 213.06 | 0.00 |
| 32 | 1 | QLAAC | 57,493 | 57,493 | 0 | 5,749.30 | 0.66 | 75.99 | 81.39 | 0.00 |
| 32 | 2 | Baseline | 66,767 | 66,767 | 0 | 6,676.70 | 0.55 | 1.32 | 211.20 | 0.00 |
| 32 | 2 | QLAAC | 58,659 | 58,659 | 0 | 5,865.90 | 0.65 | 75.14 | 79.72 | 0.00 |
| 32 | 3 | Baseline | 65,838 | 65,838 | 0 | 6,583.80 | 0.55 | 1.39 | 212.16 | 0.00 |
| 32 | 3 | QLAAC | 58,399 | 58,399 | 0 | 5,839.90 | 0.65 | 75.43 | 80.24 | 0.00 |
| 32 | 4 | Baseline | 68,666 | 68,666 | 0 | 6,866.60 | 0.53 | 1.29 | 204.10 | 0.00 |
| 32 | 4 | QLAAC | 58,422 | 58,422 | 0 | 5,842.20 | 0.65 | 73.63 | 82.10 | 0.00 |
| 32 | 5 | Baseline | 71,906 | 71,906 | 0 | 7,190.60 | 0.51 | 1.27 | 194.33 | 0.00 |
| 32 | 5 | QLAAC | 60,023 | 60,023 | 0 | 6,002.30 | 0.64 | 73.08 | 77.31 | 0.00 |
| 64 | 1 | Baseline | 69,071 | 69,071 | 0 | 6,907.10 | 0.53 | 1.29 | 438.10 | 0.00 |

| C | Trial | Policy | Attempted | 2xx | 503 | 2xx req/s | p50 | p95 | p99 | Backoff (s) |
|---|---|---|---|---|---|---|---|---|---|---|
| 64 | 1 | QLAAC | 12,595 | 12,467 | 128 | 1,246.70 | 0.59 | 3.04 | 191.93 | 541.82 |
| 64 | 2 | Baseline | 66,866 | 66,866 | 0 | 6,686.60 | 0.55 | 1.36 | 451.81 | 0.00 |
| 64 | 2 | QLAAC | 12,848 | 12,720 | 128 | 1,272.00 | 0.57 | 2.55 | 208.96 | 539.38 |
| 64 | 3 | Baseline | 66,807 | 66,807 | 0 | 6,680.70 | 0.54 | 1.40 | 452.10 | 0.00 |
| 64 | 3 | QLAAC | 18,334 | 18,206 | 128 | 1,820.60 | 0.61 | 28.51 | 128.68 | 492.30 |
| 64 | 4 | Baseline | 68,345 | 68,345 | 0 | 6,834.50 | 0.54 | 1.28 | 439.72 | 0.00 |
| 64 | 4 | QLAAC | 24,338 | 24,210 | 128 | 2,421.00 | 0.61 | 108.27 | 121.33 | 445.90 |
| 64 | 5 | Baseline | 69,195 | 69,195 | 0 | 6,919.50 | 0.53 | 1.30 | 439.14 | 0.00 |
| 64 | 5 | QLAAC | 12,099 | 11,971 | 128 | 1,197.10 | 0.59 | 3.20 | 224.86 | 540.14 |

**Table B.3:** Latency percentile and attempt aggregates (means over 5 trials; ± is sample SD)

| Client mode | C | Policy | p50 (ms) | p95 (ms) | p99 (ms) | Attempted / trial | 503 / trial |
|---|---|---|---|---|---|---|---|
| Standard | 2 | Baseline | 0.11 ± 0.01 | 0.26 ± 0.02 | 0.43 ± 0.03 | 138,067.8 ± 7,521.4 | 0.0 ± 0.0 |
| | 2 | QLAAC | 0.12 ± 0.01 | 0.28 ± 0.03 | 0.46 ± 0.06 | 132,302.2 ± 5,663.1 | 0.0 ± 0.0 |
| | 8 | Baseline | 0.53 ± 0.02 | 1.30 ± 0.05 | 28.54 ± 0.72 | 68,695.6 ± 1,968.7 | 0.0 ± 0.0 |
| | 8 | QLAAC | 0.52 ± 0.02 | 1.29 ± 0.06 | 28.02 ± 1.01 | 69,890.2 ± 2,700.5 | 0.0 ± 0.0 |
| | 32 | Baseline | 0.52 ± 0.02 | 1.31 ± 0.06 | 201.93 ± 6.43 | 69,387.0 ± 2,496.4 | 0.0 ± 0.0 |
| | 32 | QLAAC | 0.64 ± 0.01 | 72.99 ± 1.25 | 78.17 ± 1.19 | 59,884.6 ± 1,016.2 | 0.0 ± 0.0 |
| | 64 | Baseline | 0.53 ± 0.01 | 1.33 ± 0.03 | 434.75 ± 9.41 | 69,223.0 ± 1,506.2 | 0.0 ± 0.0 |
| | 64 | QLAAC | 0.65 ± 0.03 | 13.74 ± 0.52 | 393.73 ± 21.75 | 30,079.4 ± 532.4 | 17,126.8 ± 171.3 |
| Retry-aware | 2 | Baseline | 0.11 ± 0.02 | 0.25 ± 0.03 | 0.41 ± 0.04 | 141,360.6 ± 18,674.0 | 0.0 ± 0.0 |
| | 2 | QLAAC | 0.11 ± 0.01 | 0.25 ± 0.02 | 0.42 ± 0.04 | 141,154.2 ± 8,562.1 | 0.0 ± 0.0 |
| | 8 | Baseline | 0.53 ± 0.00 | 1.29 ± 0.03 | 28.40 ± 0.22 | 68,844.2 ± 784.7 | 0.0 ± 0.0 |
| | 8 | QLAAC | 0.55 ± 0.01 | 1.39 ± 0.11 | 29.49 ± 0.83 | 65,877.0 ± 2,197.0 | 0.0 ± 0.0 |
| | 32 | Baseline | 0.54 ± 0.02 | 1.34 ± 0.07 | 206.97 ± 7.90 | 67,677.0 ± 2,700.4 | 0.0 ± 0.0 |
| | 32 | QLAAC | 0.65 ± 0.01 | 74.65 ± 1.24 | 80.15 ± 1.84 | 58,599.2 ± 911.9 | 0.0 ± 0.0 |
| | 64 | Baseline | 0.54 ± 0.01 | 1.33 ± 0.05 | 444.17 ± 7.13 | 68,056.8 ± 1,160.5 | 0.0 ± 0.0 |
| | 64 | QLAAC | 0.59 ± 0.02 | 29.11 ± 45.62 | 175.15 ± 47.31 | 16,042.8 ± 5,284.6 | 128.0 ± 0.0 |

**Superseded results from the previous report revision.** The previous revision of this report was based on the sequential "tuned" runs (experiment_*_tuned_comparison.csv), produced with an earlier controller version (per docs/qlaac-controller-revision.md, CAUTION reduced keep-alive to a single request and worker saturation alone raised CAUTION). Those figures are not carried into Chapter 5. Table B.4 contrasts them with the v4 standard-client results to show how much the controller change altered the outcome; the two sets differ in controller version, run order (sequential against paired) and process lifetime, so the comparison documents history, not a controlled experiment.

**Table B.4:** QLAAC successful throughput (mean ± SD, req/s) in the superseded tuned runs and in the v4 paired standard-client runs

| C | QLAAC, tuned runs | QLAAC, v4 standard | Baseline, tuned runs | Baseline, v4 standard |
|---|---|---|---|---|
| 8 | 2,129.26 ± 50.35 | 6,989.02 ± 270.05 | 6,629.38 ± 148.21 | 6,869.56 ± 196.87 |
| 32 | 1,970.40 ± 55.83 | 5,988.46 ± 101.62 | 6,860.38 ± 107.11 | 6,938.70 ± 249.64 |
| 64 | 638.12 ± 126.44 | 1,295.26 ± 44.72 | 6,828.52 ± 157.91 | 6,922.30 ± 150.62 |

## B.5 Datasets not used for the comparison

**Table B.5:** Raw CSV files in the package that are excluded from Chapter 5, and why (data rows per file in brackets)

| File | Content | Reason for exclusion |
|---|---|---|
| benchmark_results.csv [3]; fixed_policy_8080.csv [6]; fixed_policy_high_load.csv [9]; qlaac_enabled_8081.csv [6]; qlaac_enabled_high_load.csv [9] | Legacy schema | Legacy accounting counted any "HTTP/1.1" response as success; no per-class columns; not comparable with the corrected schema (docs/experiment-methodology.md section 7) |
| benchmark_results_corrected.csv [2]; live_qlaac_*_20261018.csv [1–2 each] | Smoke tests | Single short trials used to check the harness or controller, no repetition |
| experiment_baseline_comparison.csv [8]; experiment_qlaac_comparison.csv [16] | Preliminary corrected-schema runs | Short or early runs; the QLAAC file contains trials with zero successful responses from an early controller |
| experiment_{baseline,qlaac} _tuned_comparison.csv [20 each] | Sequential 5 × 10 s runs | Previous report revision; earlier controller version (Table B.4); sequential harness |
| experiment_*_queueaware_20261019.csv, *_queueaware_v2_*.csv [20 each] | Intermediate controller revisions | Intermediate development iterations; superseded by v3 and v4 |
| experiment_*_{standard,retry-aware} _v3_20261019.csv [20 each] | Sequential runs, controller with unconditional queue-wait overload | Sequential order bias and the C = 32 collapse (Table B.6); kept as development evidence |

## B.6 Development evidence: sequential-run drift and the v3 C = 32 collapse (not used in Chapter 5)

Two findings from intermediate sequential v3 runs shaped the final design. The v3 source itself is not included in the supplied archive, so its logic is documented in docs/qlaac-final-diagnostic.md. First, baseline C = 2 fell from 21,653.36 req/s to 13,148.58 req/s 502 s later without 503s; execution order or host conditions may explain the drift, but no thermal or frequency data were recorded. Second, at C = 32 the v3 controller entered OVERLOAD with 28/64 connections queued and p95 queue wait of 257.04 ms while p95 processing latency was 0.11 ms. This indicates head-of-line delay under a moderate queue, so the final controller raises CAUTION rather than OVERLOAD in that case.

**Table B.6:** v3 sequential runs at C = 32 (QLAAC with unconditional queue-wait overload), means over 5 trials

| Run (v3, C = 32) | 2xx req/s | 503 per trial | 503 share of attempts |
|---|---|---|---|
| Baseline, standard | 6,403.86 | 0 | 0.00% |
| QLAAC, standard | 1,127.60 | 15,816 | 58.39% |
| QLAAC, retry-aware | 1,101.12 | 64 | n/a (clients back off) |

### B.7 Fresh paired benchmark rerun and archived-data comparison (21 October 2026)

**Purpose and data separation.** The complete paired performance matrix was run on a separate verification host using the compiled project server and the original benchmark runner. These measurements form a distinct verification-host replication, not a replacement for the archived v4 experiment reported in Chapter 5.

**Environment and port handling.** The rerun used Linux kernel 6.18.55 on an AMD EPYC 9V74 virtualized host, with Python 3.13.5 and GCC 14.2.0. The runtime reported five available CPUs, while the cgroup CPU quota was 400000/100000 (four CPU-equivalents). An unrelated service was already listening on port 8080. The server was therefore smoke-tested on 127.0.0.1:18080: /health returned HTTP 200 with the expected HTTP-Server-C17/1.0.0 identity and HTTP/1.1 protocol, and /metrics returned successfully.

**Readiness-check issue and mitigation.** The original runner checked TCP connectability, which alone cannot establish that the expected application is responding. For the recorded run, port 8080 was avoided because another service was listening there; the project server was started on port 18080, and its /health and /metrics endpoints were checked successfully before the benchmark matrix. An application-level readiness refinement was tested separately after the full run and is described in B.8; it was not used to start the recorded 80-trial experiment.

Reproduction command. From the project root, after a clean build, the full run was executed with the supplied paired benchmark script using:

```
python3 -u benchmark/run_reproducible_evaluation.py --mode both --client-mode both
--port 18080 --duration 10 --trials 5 --tag fresh_rerun_20261021
```

**Completion and integrity evidence.** The runner emitted completion markers for both STANDARD and RETRY-AWARE modes. Four fresh per-trial CSV files each contain 20 rows (C = 2, 8, 32, 64, with trial IDs 1–5 at every concurrency), for 80 timed 10-second trials total. The four diagnostic timeline CSVs were also saved. All CSV checks passed: status-class counts reconcile to completed HTTP responses, successful_rps agrees with successful 2xx responses divided by trial duration, and measurements are finite. Eighty per-process server logs show the expected four-worker configuration, the intended QLAAC enabled/disabled split, and complete teardown; no fatal or bind errors were found. The clean build succeeded. SHA-256 comparisons found no changes to the archived v4 CSVs or selected source-code files.

**Table B.7: Fresh verification-host throughput and HTTP 5xx accounting (mean ± sample SD over five trials)**

| Client mode | C | Baseline 2xx RPS | QLAAC 2xx RPS | Δ throughput | QLAAC 5xx total |
|---|---|---|---|---|---|
| Standard | 2 | 37,065.64 ± 1,233.08 | 36,254.12 ± 997.66 | -2.2% | 0 |
| Standard | 8 | 24,702.80 ± 2,083.31 | 25,966.64 ± 973.23 | +5.1% | 0 |
| Standard | 32 | 26,315.90 ± 981.51 | 22,846.24 ± 867.64 | -13.2% | 0 |
| Standard | 64 | 26,205.88 ± 846.95 | 4,662.02 ± 73.38 | -82.2% | 276,337 |
| Retry-aware | 2 | 36,594.26 ± 1,206.41 | 36,727.30 ± 1,786.18 | +0.4% | 0 |
| Retry-aware | 8 | 25,284.54 ± 559.12 | 25,267.98 ± 2,395.09 | -0.1% | 0 |
| Retry-aware | 32 | 26,100.96 ± 473.80 | 23,295.10 ± 569.28 | -10.8% | 0 |
| Retry-aware | 64 | 25,891.42 ± 667.90 | 8,727.14 ± 167.96 | -66.3% | 640 |

Notes: 2xx RPS counts successful HTTP 2xx responses per second. Throughput change is (QLAAC − baseline) / baseline × 100. The 5xx total is summed over the five QLAAC trials for that mode and concurrency. At C = 64, standard-client QLAAC recorded 276,337 5xx responses in total (about 54.24% of attempts); retry-aware QLAAC recorded 640 in total (128 per trial, about 0.15% of attempts) and a mean 452.48 client-thread-seconds of Retry-After back-off per trial.

**Table B.8: Fresh verification-host p95 and p99 latencies (ms; mean ± sample SD over five trials)**

| Client mode | C | Base p95 | QLAAC p95 | Base p99 | QLAAC p99 | p99 reduction |
|---|---|---|---|---|---|---|
| Standard | 2 | 0.100 ± 0.007 | 0.102 ± 0.004 | 0.202 ± 0.022 | 0.202 ± 0.019 | +0.0% |
| Standard | 8 | 0.460 ± 0.084 | 0.418 ± 0.023 | 7.766 ± 0.676 | 7.390 ± 0.239 | +4.8% |
| Standard | 32 | 0.418 ± 0.025 | 17.640 ± 0.620 | 52.716 ± 1.816 | 21.944 ± 0.979 | +58.4% |
| Standard | 64 | 0.414 ± 0.018 | 2.658 ± 0.161 | 113.210 ± 2.423 | 90.162 ± 7.858 | +20.4% |
| Retry-aware | 2 | 0.098 ± 0.004 | 0.100 ± 0.007 | 0.200 ± 0.017 | 0.198 ± 0.029 | +1.0% |
| Retry-aware | 8 | 0.428 ± 0.018 | 0.444 ± 0.093 | 7.588 ± 0.111 | 7.662 ± 0.748 | -1.0% |
| Retry-aware | 32 | 0.414 ± 0.009 | 17.370 ± 0.455 | 52.868 ± 0.586 | 21.476 ± 0.481 | +59.4% |
| Retry-aware | 64 | 0.418 ± 0.015 | 26.858 ± 1.280 | 114.790 ± 2.968 | 33.424 ± 1.308 | +70.9% |

Note: p99 reduction = (baseline mean p99 − QLAAC mean p99) / baseline mean p99 × 100. Positive values indicate lower QLAAC p99; negative values indicate higher QLAAC p99.

**Interpretation against archived v4.** The fresh run reproduces the broad C = 32 trade-off in both client modes: successful throughput fell while p99 improved by 58.4% (standard) and 59.4% (retry-aware), without 5xx shedding. However, the p95 tail increased from 0.418 to 17.640 ms in standard mode and from 0.414 to 17.370 ms in retry-aware mode. At C = 64, standard-mode throughput fell 82.2%, 54.24% of attempts were shed, and p99 improved 20.4%; with retry-aware clients, throughput fell 66.3%, p99 improved 70.9%, and clients backed off after Retry-After.

**These percentages are not identical to the archived v4 results in Chapter 5.** The original v4 results report C = 32 p99 reductions of 61.3% with throughput decreases of 13.7% (standard) and 13.4% (retry-aware); at C = 64 they report a standard-mode throughput decrease of 81.3% with 9.4% p99 reduction, and a retry-aware throughput decrease of 76.6% with 60.6% p99 reduction. Baseline successful throughput is much higher on this verification host, and the archived v4 files do not contain enough original host/compiler/thermal metadata to normalize these runs. Therefore these datasets are not pooled, and Chapter 5 continues to present archived v4 as its primary experiment; Tables B.7–B.8 document this separate fresh rerun.

**Artifact locations.** The verification package contains the four fresh CSVs, four diagnostic timeline CSVs, complete benchmark console log, 80 server-process logs, clean-build log, smoke-test outputs, environment record, statistics-verifier outputs for fresh and archived v4 data, row/accounting validation summary, SHA-256 manifests and the separately tested readiness-check patch. The original uploaded source ZIP and archived measurements are preserved unchanged.

### B.8 Readiness-check validation and regression testing (21 October 2026)

**Scope and relation to the performance run.** The readiness check was refined after the fresh paired benchmark run in B.7. This is a startup-validation change, not a new performance experiment. The archived v4 measurements, fresh rerun CSVs, logs and Chapter 5 results remain unchanged; the 80-trial performance benchmark was not rerun after this refinement.

**Implementation.** In benchmark/run_reproducible_evaluation.py, wait_for_server() sends GET /health and requires HTTP 200, valid JSON, status “healthy”, server identity “HTTP-Server-C17/1.0.0”, and protocol “HTTP/1.1”. It uses time.monotonic(), caps each HTTPConnection blocking timeout to the smaller of one second and the remaining polling budget, and caps retry sleep to the remaining budget. These limits bound individual blocking operations and retry intervals; they do not guarantee an exact end-to-end wall-clock cutoff if a peer slowly trickles a response. Expected transport, HTTP and JSON errors are handled explicitly.

**Regression coverage.** tests/test_readiness_check.py covers valid health responses, unrelated HTTP/HTML services, non-HTTP listeners, non-200 responses, malformed JSON, missing fields, unavailable and slow servers, transient retries, and a live compiled C server. The focused readiness-check run passed. The complete suite passed 44/44 C and 48/48 Python tests (92 total); git diff --check also passed. The current project repository includes the readiness-check implementation and regression tests. The earlier archived source bundle predates these additions; use the current repository revision for the complete 92-test suite.

**Interpretation.** A TCP listener alone can be mistaken for the project server. The revised check validates the application-level health response and uses a monotonic polling budget. Passing these tests increases confidence in benchmark startup validation, but does not establish that historical and fresh performance timings are identical; the distinct results and limitations remain documented in B.7.